

UNIVERSITY OF MALTA

Faculty of Engineering

Department of Systems and Control Engineering

FINAL YEAR PROJECT

B.ENG. (Hons)

Identifying Optimal Investment Strategies with Deep Reinforcement Learning

by

Mia GAUCI

A dissertation submitted in partial fulfilment
of the requirements of the award of

Bachelor of Engineering (Hons.) of the University of Malta

Copyright Notice

1. Copyright in text of this dissertation rests with the Author. Copies (by any process) either in full, or of extracts may be made only in accordance with regulations held by the Library of the University of Malta. Details may be obtained from the Librarian. This page must form part of any such copies made. Further copies (by any process) made in accordance with such instructions may not be made without the permission (in writing) of the Author.
2. Ownership of the right over any original intellectual property which may be contained in or derived from this dissertation is vested in the University of Malta and may not be made available for use by third parties without the written permission of the University, which will prescribe the terms and conditions of any such agreement.



**L-Università
ta' Malta**

FACULTY/INSTITUTE/CENTRE/SCHOOL Engineering

DECLARATION BY UNDERGRADUATE STUDENTS

(a) Authenticity of Long Essay/Dissertation

I hereby declare that I am the legitimate author of this Long Essay/Dissertation and that it is my original work.

No portion of this work has been submitted in support of an application for another degree or qualification of this or any other university or institution of higher education.

I hold the University of Malta harmless against any third party claims with regard to copyright violation, breach of confidentiality, defamation and any other third party right infringement.

(b) Research Code of Practice and Ethics Review Procedures

I declare that I have abided by the University's Research Ethics Review Procedures. Research Ethics & Data Protection form code **ENG-2024-00097**

Identifying Optimal Investment Strategies with Deep Reinforcement Learning

Abstract

The rise of fully automated trading systems has transformed global financial markets, placing greater emphasis on intelligent data-driven decision making. This thesis explores the development of optimal investment strategies using Deep Reinforcement Learning (DRL), with a particular focus on the Proximal Policy Optimisation (PPO) algorithm. Historical closing price data from a diversified portfolio of seven technology stocks was collected, processed and combined with market indicators to form the model inputs. A supervised learning baseline was first established using a Multilayer Perceptron (MLP) to provide a performance benchmark. Subsequently, DRL agents that incorporate different neural network architectures, MLPs, Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN), were implemented within a custom PPO framework designed for multiple stock portfolio management. Each model was evaluated using two state representations: normalised closing prices and normalised engineered market features, enabling a comparison of model performance under varying input dimensions. The evaluation was carried out using Monte Carlo rollouts in a custom simulated trading environment using 2023 test data. The PPO agent with an MLP architecture and engineered features achieved the most stable returns, averaging a gain of 152%, while the CNN-based agent with closing price-only input reached a maximum return of 265% but with a higher volatility. These results suggest that, when the models are appropriately structured and trained, DRL agents can outperform both traditional supervised learning approaches and passive strategies in simulated markets, offering a promising foundation for further research into adaptive algorithmic portfolio optimisation.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Kenneth Scerri, and my co-supervisor, Dr. Liam Butler, for their invaluable guidance and support throughout this project.

Contents

Copyright Notice	i
Declaration of Authorship	ii
Abstract	iii
Acknowledgements	iv
Contents	v
List of Figures	ix
List of Tables	xi
List of Acronyms	xii
1 Introduction	1
1.1 Aims and Objectives	2
1.2 Structure of Report	3
2 Literature Review	4
2.1 Machine Learning	4
2.2 Artificial Neural Networks	5
2.2.1 Shallow Neural Networks	5
2.2.2 Deep Neural Networks	6
2.3 Function Approximation Techniques	7
2.3.1 Multilayer Perceptrons	7
2.3.2 Convolutional Neural Networks	8
2.3.3 Recurrent Neural Networks	9
2.4 Reinforcement Learning	10
2.4.1 Markov Decision Process	12
2.4.1.1 Markov Chains	12
2.4.1.2 Markov Reward Process	13

2.4.1.3	Markov Decision Process	13
2.4.1.4	Exploration vs. Exploitation	13
2.4.2	Bellman Equation	14
2.4.3	Reinforcement Learning Algorithms	15
2.4.3.1	Function Approximation Techniques in Reinforcement Learning	16
2.5	Deep Reinforcement Learning	16
2.5.1	Model-Based Reinforcement Learning	18
2.5.2	Model-Free Reinforcement Learning	18
2.5.3	Value-Based Methods	19
2.5.3.1	Q-Learning	20
2.5.3.2	Deep Q-Learning	20
2.5.4	Policy-Based Methods	21
2.5.4.1	Policy Gradient Methods	21
2.5.4.2	Policy-Based Methods vs Value-Based Methods	22
2.5.5	Actor-Critic Methods	22
2.5.5.1	Deterministic Policy Gradient	23
2.5.5.2	Deep Deterministic Policy Gradient	24
2.5.5.3	Proximal Policy Optimisation	24
2.5.6	Special Cases of Actor-Critic Methods	25
2.5.6.1	Actor-Only Method	25
2.5.6.2	Critic-Only Method	25
2.6	Conclusion	26
3	Data	27
3.1	Data Source Selection	27
3.1.1	Data Acquisition	28
3.2	Security Selection	29
3.2.1	Overview of Selected Stocks	31
3.2.2	Corporate Actions Adjustments	32
3.2.3	Impact of Market Events	33
3.3	Market Indicators	34
3.3.1	Feature Engineering	34
3.3.2	Indicator Limitations	35
3.4	Data Preprocessing	35
3.4.1	Data Integrity	35
3.4.2	Data Normalisation	36
3.4.3	Final Data Format	37
3.5	Buy-and-Hold Threshold	38
3.6	Conclusion	38
4	Methodology	39
4.1	Software Infrastructure	39
4.1.1	Data Structure	40

4.2	Supervised Learning Implementation	41
4.2.1	Data Preprocessing	41
4.2.2	Model Development	42
4.2.2.1	Temporal Window Configuration	43
4.2.3	Evaluation Metrics and Performance Analysis	44
4.2.4	Simulated Market Evaluation	45
4.3	Reinforcement Learning Implementation	47
4.3.1	Data Preprocessing	48
4.3.1.1	Experimental Data Structure	49
4.3.2	System Overview	49
4.3.2.1	Reward Function Design	51
4.3.2.2	Stochastic Action Sampling for Portfolio Allocation	51
4.3.3	Proximal Policy Optimisation Agent Architecture	52
4.3.3.1	Actor–Critic Network Overview	52
4.3.4	Proximal Policy Optimisation Training Framework	53
4.3.4.1	Advantage Estimation and Critic Loss Computation	53
4.3.4.2	Policy Update	55
4.3.4.3	Training Algorithm Summary	56
4.4	Neural Network Architecture Comparison	56
4.4.1	Multilayer Perceptron Architecture	57
4.4.1.1	With Closing Price as the State	57
4.4.1.2	With Features as the State	59
4.4.2	Convolutional Neural Network Architecture	60
4.4.2.1	With Closing Price as the State	61
4.4.2.2	With Features as the State	62
4.4.3	Recurrent Neural Network	64
4.4.3.1	With Closing Price as the State	64
4.4.3.2	With Features as the State	66
4.4.4	Summary of Model Tuning	67
4.4.5	Simulated Market Evaluation	68
4.4.6	Full System Integration	71
4.5	Conclusion	72
5	Results and Discussion	73
5.1	Overview of Evaluation Approach	73
5.2	Model Performance on Training Data	73
5.3	Model Performance on Testing Data	76
5.3.1	Risk–Reward Classification	79
5.3.2	Computational Considerations	79
5.4	Comparison with Supervised Learning Implementation	80
5.5	Conclusion	81
6	Conclusion	82
6.1	Project Overview	82

6.2	Future Developments	83
6.3	Future Research	84
6.4	Final Remarks	85

Bibliography	86
---------------------	-----------

List of Figures

2.1	The MLP architecture (adopted from [1])	7
2.2	The CNN architecture (adopted from [1])	8
2.3	The RNN architecture showing the feedback loop mechanism (adopted from [2])	10
2.4	Agent-Environment interaction in an MDP (adopted from [3])	12
2.5	Tree diagram of different RL algorithm categories.	17
2.6	The actor-critic architecture (adopted from [3])	23
3.1	Violin plot comparing the daily return distributions for the selected stocks.	31
3.2	Effect of Apple Stock Split on Price Data: Unadjusted Closing Prices Showing Apparent Price Drop (Left) and Adjusted Closing Prices Reflecting True Continuity (Right)	32
3.3	Closing prices of selected technology stocks from 2018 to 2022. The red region marks the COVID-19 crash (February 2020 to March 2020), and the green region indicates the <i>tech rally</i> (April 2020 to November 2021).	33
3.4	Visualisation of the different normalisation methods for Apple (2022-2023).	36
3.5	Buy-and-Hold portfolio growth.	38
4.1	Raw closing prices for the selected stocks used in this study, with the training period shaded in yellow and the testing period shaded in pink.	40
4.2	Histogram of daily closing price differences for AAPL.	41
4.3	Model performance with different window sizes compared to the buy-and-hold strategy	44
4.4	Confusion matrix for AAPL, showing the number of correct and incorrect predictions for each action class.	44
4.5	Portfolio value over time for the supervised MLP model compared to the buy-and-hold strategy.	46
4.6	Z-score Normalised Closing Prices for Selected Stocks	49
4.7	Reinforcement Learning Architecture: Agent–Environment Interaction	50
4.8	Architecture of the MLP Actor–Critic Network	57
4.9	Actor loss (left) and average log-probability (right) for MLP architecture trained on closing price input.	59
4.10	Actor loss (left) and average log-probability (right) for the MLP architecture with market features as input.	60
4.11	Architecture of the CNN Actor–Critic Network	60
4.12	Actor loss (left) and average log-probability (right) for CNN architecture trained on closing price input.	62

4.13	Actor loss (left) and average log-probability (right) for CNN architecture trained with market features input.	63
4.14	Architecture of the RNN Actor–Critic Network	64
4.15	Actor loss (left) and average log-probability (right) for RNN architecture trained on closing price input.	66
4.16	Actor loss (left) and average log-probability (right) for RNN architecture trained with market features input.	67
4.17	Under performing portfolio value vs buy-and-hold.	71
4.18	Over performing portfolio value vs buy-and-hold.	71
5.1	Violin Plot of the final portfolio values for 200 Monte Carlo rollouts on Training data for each model	74
5.2	Violin Plot of the final portfolio values for 200 Monte Carlo rollouts on Testing data for each model	77
5.3	Median portfolio value trajectories from 200 Monte Carlo runs for PPO agents using MLP, CNN, and RNN architectures with engineered features as input, compared against a supervised MLP baseline and the buy-and-hold strategy during the 2023 test period.	81

List of Tables

3.1	Sample structure of downloaded stock data using daily intervals	29
3.2	Overview of the eleven Stock Market Sectors	30
3.3	Summary of Selected Technology Stocks	31
3.4	Example of 3-Day Rolling Window for calculated Features	37
4.1	Action Categories Based on Price Change	42
4.2	MLP Hyperparameters Used for Supervised Learning	43
4.3	Overview of the Actor and Critic Neural Network Architecture	52
4.4	MLP using Closing Price PPO Agent Hyperparameters	58
4.5	MLP using Features PPO Agent Hyperparameters	59
4.6	CNN using Closing Price PPO Agent Hyperparameters	61
4.7	CNN using Features PPO Agent Hyperparameters	63
4.8	RNN using Features PPO Agent Hyperparameters	65
4.9	RNN using Features PPO Agent Hyperparameters	66
4.10	Convergence and Policy Stabilisation Metrics for PPO Architectures . .	68
4.11	Example of Action Execution Across Two Trading Days	69
5.1	Training-Time Performance Summary Across 200 Monte Carlo Rollouts	74
5.2	Testing-Time Performance Summary Across 200 Monte Carlo Rollouts .	76
5.3	Risk–Reward Classification of PPO Models on Testing Data	79
5.4	Training Duration by Model	80

List of Acronyms

FinTech

Financial Technology. [1](#)

AI

Artificial Intelligence. [1](#), [4](#)

RL

Reinforcement Learning. [1–4](#), [7–18](#), [20](#), [22](#), [24–26](#), [29–31](#), [33](#), [35](#), [38](#), [39](#), [41](#), [43](#), [47–49](#), [51](#), [53](#), [54](#), [57](#), [58](#), [68](#), [73](#)

DL

Deep Learning. [2](#), [5](#), [7](#), [18](#), [20](#), [38](#)

DRL

Deep Reinforcement Learning. [2](#), [4](#), [16](#), [17](#), [24](#), [26](#), [39](#), [40](#), [49](#), [82](#)

PPO

Proximal Policy Optimisation. [2](#), [19](#), [24](#), [39](#), [47–49](#), [51–53](#), [55–62](#), [65](#), [67](#), [68](#), [71–73](#), [76](#), [78](#), [80–82](#)

ML

Machine Learning. [2–5](#), [10](#), [26](#), [38](#), [39](#)

SVM

Support Vector Machines. [4](#)

DNN

Deep Neural Network. [5](#), [6](#), [16–18](#), [20](#)

ANN

Artificial Neural Network. [5](#), [9](#)

SNN

Shallow Neural Network. [5](#)

ReLU

Rectified Linear Unit. [6](#), [7](#), [60](#), [64](#)

MLP

Multilayer Perceptron. [6](#), [7](#), [16](#), [41](#), [42](#), [45–47](#), [52](#), [56](#), [57](#), [59](#), [61](#), [62](#), [79](#)

CNN

Convolutional Neural Network. [6](#), [8](#), [9](#), [16](#), [52](#), [56](#), [60–62](#), [64](#), [79](#)

RNN

Recurrent Neural Network. [6](#), [9](#), [10](#), [16](#), [52](#), [56](#), [64–67](#), [79](#)

LSTM

Long Short-Term Memory. [10](#), [65](#), [84](#)

MDP

Markov Decision Process. [12–16](#), [26](#)

GPI

Generalized Policy Iteration. [15](#)

TD

Temporal Difference. [15](#), [16](#), [53](#)

PG

Policy Gradient. [16](#)

DQN

Deep Q-Networks. [17–21](#), [24](#), [25](#)

TRPO

Trust Region Policy Optimisation. [22](#)

DPG

Deterministic Policy Gradient. [23](#), [24](#)

DDPG

Deep Deterministic Policy Gradient. [23](#), [24](#)

API

Application Programming Interface. [27](#), [28](#), [31](#), [84](#), [85](#)

CSV

comma-separated values. [29](#), [37](#)

EMA

Exponential Moving Average. [34](#)

RSI

Relative Strength Index. [34](#), [35](#)

MACD

Moving Average Convergence Divergence. [34](#), [35](#)

GAE

Generalised Advantage Estimation. [53](#)

SGD

Stochastic Gradient Descent. [54](#), [64](#), [66](#)

GRU

Gated Recurrent Units. [65](#), [84](#)

Chapter 1

Introduction

The rise of [Financial Technology \(FinTech\)](#) has changed the way the stock market operates, with automation and data-driven decision-making becoming central to modern trading systems. In today's financial markets, algorithmic trading has increasingly replaced human traders, with automated systems executing the majority of trades. More than 70% of the trades in the United States stock market are now handled by algorithms [4]. These algorithms are capable of analysing large volumes of market data in real time and make quick decisions based on patterns learnt from historical data that surpass what human traders can achieve.

Traditionally, trading systems relied on rule-based approaches, where decisions were made according to a fixed set of pre-programmed rules or conditions. However, the stock market is naturally volatile, with continuous data shifts and rapidly changing market conditions. As a result, rule-based approaches often require frequent updates and re-tuning to stay relevant, as they cannot adapt to new data or unexpected market events without human intervention.

However, with the rise of [Artificial Intelligence \(AI\)](#), companies are eager to adopt technologies that can continuously learn and refine strategies. These [AI](#) systems allow algorithms to learn from past data, adapt to new market conditions, and optimise trading strategies autonomously. For instance, in 2017, JPMorgan's *LOXM* [AI](#) tool demonstrated the ability to execute trades with minimal slippage, providing a competitive edge in volatile markets [5]. This shift brings to light the growing dependence on [AI](#) not only for trade execution but also for tasks such as market analysis, portfolio management, and risk assessment.

Among [AI](#) technologies that are gaining traction, [Reinforcement Learning \(RL\)](#) [6–8] stands out for its ability to improve decision making over time. [RL](#) models continuously learn from market feedback, allowing them to adjust strategies in real time.

This adaptability makes [RL](#) particularly suitable for tasks such as portfolio optimisation, where strategies must be constantly updated in response to new information. Leading financial institutions such as Goldman Sachs and DE Shaw are already applying [RL](#) to their trading systems. Goldman Sachs has used [RL](#) in high-frequency trading (HFT) [9] to optimise trade executions, reduce market impact, and improve liquidity provision. Similarly, DE Shaw employs [RL](#) to refine its algorithmic trading systems, using [Deep Learning \(DL\)](#) networks that improve over time based on continuous market feedback [10]. Such cases illustrate the growing potential of [RL](#) in the future of financial systems.

1.1 Aims and Objectives

This research aims to explore the application of [Deep Reinforcement Learning \(DRL\)](#), specifically the [Proximal Policy Optimisation \(PPO\)](#) algorithm, in the development and optimisation of investment strategies for a group of seven stocks.

To achieve the objectives of this research, the following areas were addressed:

- A detailed review of the foundational concepts of [Machine Learning \(ML\)](#), [RL](#), and [DRL](#).
- A study of the stock market data for time-series financial analysis and how to read and store stock prices for usage.
- The development of a baseline supervised learning model to establish initial trading benchmarks.
- The design and implementation of the [PPO](#) algorithm within an actor-critic framework.
- An evaluation of different neural network architectures, including hyperparameter tuning and model-specific training strategies.
- Building a custom investment [RL](#) environment that simulates realistic market conditions and supports multiple stock trading with continuous action spaces.
- An evaluation and comparison of six different [DL](#) methods, tested over one year with an initial portfolio of \$10,000.

1.2 Structure of Report

This report is organised into six chapters:

Chapter 2 provides background on the core theories and summarizes the main literature in [ML](#) and [RL](#). It introduces the basic concepts and describes different [RL](#) approaches.

Chapter 3 describes the data used in this study, including how it was collected and processed. It also covers the chosen features and the format of the dataset used to train and evaluate the models.

Chapter 4 details the methodology adopted in the project. Describing the implementation of both supervised and [RL](#) models, including model architecture, training procedures, and evaluation techniques applied to the portfolio allocation task.

Chapter 5 presents the results and discussion that evaluates the performance of the model in multiple runs, compares the findings with traditional baselines, and analyses the results to identify the strengths and limitations of each method.

Chapter 6 concludes the report with a summary of findings and outlines directions for future research.

Chapter 2

Literature Review

This chapter provides an overview of [ML](#) and [RL](#), covering their foundational concepts and the progression to [DRL](#). The theoretical principles discussed form the basis for the methods implemented in Chapter 4.

2.1 Machine Learning

[ML](#) is a fundamental building block of [AI](#), where systems can automatically learn from data and improve performance without direct programming [11, 12]. Traditionally, statistical methods such as linear regression [13] and decision trees [14, 15] have been used for predictive modelling. Given the growing intricacy of modern data, [ML](#) has introduced algorithms capable of recognising intricate patterns and making intelligent predictions, surpassing the limitations of traditional statistical models [16]. [ML](#) is further divided into three main learning approaches:

- Supervised learning, which is based on labelled data to train models [17],
- Unsupervised learning, that identifies patterns in unlabelled data [17], and
- [RL](#), that optimises decision-making using trial-and-error interactions with the environment [3, 11, 18].

One of the most impactful applications of [ML](#) has been in financial markets [17], where algorithms evaluate large amounts of current and historical data to optimise investment strategies. Techniques such as time series forecasting [14] have been widely employed to predict asset price movements, while [Support Vector Machines \(SVM\)](#) [19, 20] are used for risk assessment [21]. However, traditional [ML](#) models often struggle to capture intricate relationships between multiple variables, limiting their predictive capabilities.

To address these challenges, DL has emerged as an extension of ML, which uses multilayered Deep Neural Networks (DNNs) [22] to model non-linear dependencies within the datasets [3]. Unlike conventional ML algorithms, DL models can automatically extract hierarchical features, which is particularly advantageous in high-dimensional applications such as algorithmic trading and financial risk modelling [23]. This progression from traditional statistical models to ML and ultimately DL has enabled powerful predictive and decision-making systems, driving innovation across various domains, including healthcare and automation [3].

2.2 Artificial Neural Networks

Artificial Neural Networks (ANNs), commonly referred to as neural networks, are widely used to identify non-linear relationships in financial data [3]. Unlike traditional models, ANNs are made up of interconnected layers of neurons whose purpose is to process information through weighted connections, allowing adaptive learning from vast datasets [24]. The fundamental structure of neural networks includes an input layer, one or more hidden layers, and an output layer, enabling them to model dependencies that are often missed by conventional statistical techniques [25]. ANNs can be broadly categorised into Shallow Neural Networks (SNNs) and DNNs.

2.2.1 Shallow Neural Networks

One of the most basic types of neural network is SNNs, which are used for simple pattern recognition applications. These networks convert input data using activation functions, which are mathematical functions that decide whether or not to activate a neuron. However, because of their shallow architecture, they cannot learn intricate hierarchical patterns [22]. Despite this, they have been successfully used in applications such as the prediction of stock price movements based on historical price trends, where relationships between a few key indicators can be captured by the model [26]. However, when dealing with high-dimensional financial data that involve multiple interacting variables, shallow networks often fail to perform reliably, as they struggle to capture detail in the non-linear relationships of the data and learn pattern between market indicators over a long period of time, necessitating the transition to deeper architectures [27].

2.2.2 Deep Neural Networks

DNNs build upon conventional models by incorporating several hidden layers, which allow them to automatically learn and extract layered features from large-scale datasets [22, 24, 27]. One of the biggest advantages of **DNNs** is the fact that they can automatically extract features, thus eliminating the need for direct programming and changing the model features [22]. This capability is particularly beneficial in financial modelling, where markets exhibit non-linear, high-dimensional relationships that require sophisticated learning architectures [28].

DNNs can represent intricate data patterns by incorporating non-linearity through activation functions such as the **Rectified Linear Unit (ReLU)** [3]. They use back propagation, a supervised learning approach that iteratively modifies the network's weights, to improve their predictions. This approach involves calculating the loss function's gradient for each parameter and adjusting the weights through gradient descent, enabling the model to reduce errors and enhance performance across successive training cycles. These properties enable them to learn sophisticated functions that map financial indicators to investment strategies [24].

Despite their effectiveness, **DNNs** require substantial computational resources and are prone to over-fitting, particularly when trained on limited data. Techniques such as dropout regularisation [29], batch normalisation [30], and transfer learning [31] help mitigate these issues, improving performance in different financial scenarios [25]. Within the broader category of **DNNs**, numerous specialised architectures have been developed to address various tasks, including but not limited to the following;

- **Multilayer Perceptrons (MLPs)** are fully connected neural networks used for general-purpose learning,
- **Convolutional Neural Networks (CNNs)** are designed to process spatial data by extracting hierarchical features, and
- **Recurrent Neural Networks (RNNs)** are used to model time-dependent and sequential data.

2.3 Function Approximation Techniques

2.3.1 Multilayer Perceptrons

MLPs remain one of the most studied and applied architectures in DL. MLPs are fundamentally composed of an input layer, one or more fully connected hidden layers, and an output layer, as illustrated in Figure 2.1. According to the Universal Approximation Theorem, an MLP with just one hidden layer can approximate any continuous function within a bounded domain [32], yet practical applications routinely employ multiple hidden layers to achieve more compact representations and improved sample efficiency. In RL, this hierarchical stacking of linear transformations and non-linear activations enables agents to extract increasingly abstract features from raw inputs, whether the input is pixels, price series, or engineered indicators, without the need for manual feature engineering [22].

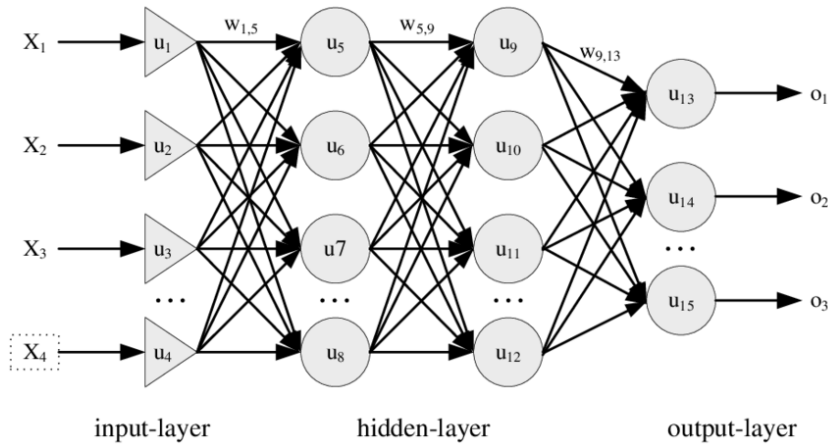


FIGURE 2.1: The MLP architecture (adopted from [1])

Despite their conceptual simplicity, the performance of MLPs is dependent on the architectural and optimisation choices. Early work identified the vanishing-gradient problem as a key barrier to training deeper MLPs since gradients propagated backwards through many layers tend to shrink, preventing lower layers from learning [33]. The adoption of ReLU activations, with their non-saturating gradient for positive inputs, significantly mitigated this issue and allowed networks beyond two hidden layers to reliably converge on large-scale tasks [34]. Complementary advances, such as dropout regularisation [29], batch normalisation [30], and adaptive optimisers like Adam [35], have further improved learning and training stability, making the three or four-layer MLPs a common choice in both on-policy and off-policy RL algorithms [36].

2.3.2 Convolutional Neural Networks

Building on DL concepts, **CNNs** play a key role in the extraction of features and data representation, particularly in **RL** tasks that require managing detailed and unstructured input data. Unlike fully connected standard networks, **CNNs** apply convolutional filters to detect local patterns, progressively building more detailed representations in deeper layers using spatial hierarchies [37].

The fundamental elements of a **CNN**, as shown in Figure 2.2, include

- the **convolutional layers** process the input data using multiple adjustable filters, allowing them to recognise patterns such as edges, textures, and other detailed features. The mathematical operation performed is defined as:

$$X_{ij}^l = \sum_{m,n} W_{mn}^l X_{(i+m)(j+n)}^{l-1} + b^l \quad (2.1)$$

where W represents the filter weights, X denotes the input feature map, l refers to the length of the dataset and b refers to the bias term. Both W and b will retain these definitions throughout the remainder of the chapter,

- the **pooling layers** that reduce spatial dimensions, preserve essential information while lowering computational demands:

$$X_{ij}^l = \max_{m,n} (X_{(i+m)(j+n)}^{l-1}) \quad (2.2)$$

- **fully connected layers** which once feature extraction is complete, the high-level representations are flattened and passed through dense layers to output predictions or policy decisions.

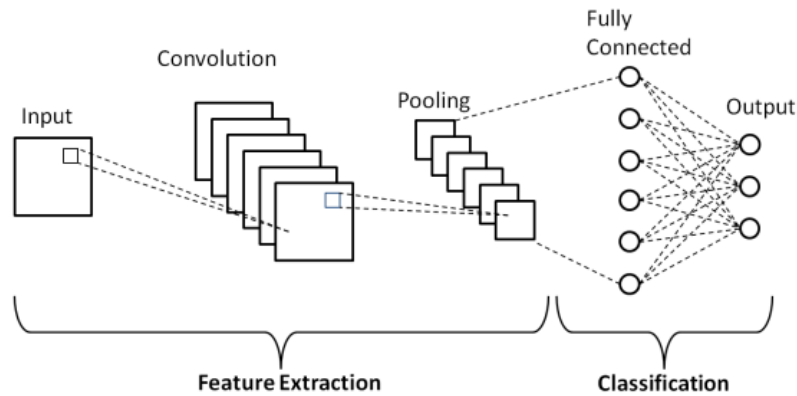


FIGURE 2.2: The CNN architecture (adopted from [1])

CNNs are frequently applied in policy-based and actor-critic RL methods to enhance feature extraction and improve policy stability [38]. Their strength lies in capturing spatial and temporal patterns, making them well suited for financial time series.

2.3.3 Recurrent Neural Networks

RNNs represent a type of ANN specifically developed to handle sequential data by preserving past inputs through recurrent connections. In contrast to conventional feed-forward networks that process each input separately, RNNs use feedback loops to retain information over multiple time steps. This recurrent structure makes them suitable for tasks that involve temporal dependencies [39].

An RNN uses the current input x_t and the previous hidden state h_{t-1} to update its hidden state h_t at each time step t . RNNs are able to capture and save information from previous time steps, which are mathematically represented as:

$$h_t = \sigma(W_h x_t + U_h h_{t-1} + b_h) \quad (2.3)$$

where:

- σ represents a non-linear activation function, such as the hyperbolic tangent (tanh) or ReLU,
- W_h are the weight matrices of the input states, and
- U_h are weight matrices of the hidden states.

The output y_t is computed at time t :

$$y_t = \text{softmax}(W_y h_t + b_y) \quad (2.4)$$

This structure enables RNNs to handle sequential patterns by learning temporal correlations between past and present states. Figure 2.3 shows the layout of a basic RNN, where the hidden states are recurrently connected across time steps, allowing the information to flow through sequences.

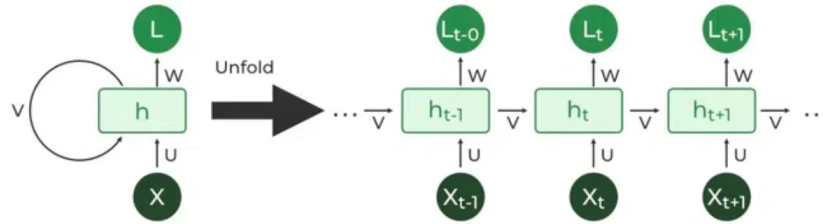


FIGURE 2.3: The RNN architecture showing the feedback loop mechanism (adopted from [2])

Although this structure allows the model to handle sequential data, conventional **RNNs** face two major challenges:

- the **exploding gradient problem**, which occurs when gradients grow exponentially, resulting in unstable weight updates and poor convergence during training, and
- the **vanishing gradient problem**, where gradients shrink as they are propagated backward through many time steps, reducing the model's ability to learn long-term relationships.

These limitations reduce the ability of standard **RNNs** for tasks that require long-term memory retention, as the network struggles to propagate useful information over extended sequences. To overcome these issues, specialised architectures such as **Long Short-Term Memory (LSTM)** networks [40] have been developed [39].

2.4 Reinforcement Learning

As discussed in Section 2.1, **RL** is one of the three main learning approaches in **ML**, where an agent improves its decision making by continuously interacting with its environment to gain the highest possible rewards over time [3]. The **RL** framework comprises two primary entities:

- the **agent** is the system or entity that makes decisions by observing its environment and taking actions. It learns through rewards and penalties, adjusting its approach over time to improve long-term outcomes, and
- the **environment** refers to the external system with which the agent interacts and responds through its actions [41].

As the agent interacts with the environment, a continuous feedback loop is created, helping it improve its decision-making over time. This process is defined by three main

elements: states (S), which represent the environment's current condition; actions (A), which are the possible choices the agent can take to affect the environment; and rewards (R), which provide feedback based on the outcomes of the agent's actions [3, 42, 43]. The agent aims to develop a policy (π), which is a strategy that helps it decide which actions to take in different states to earn the highest possible rewards over time [44]. To do this, it uses a value function (V) that estimates the total rewards it can expect from a given state, allowing it to make better decisions for long-term success.

In [RL](#) the concept of delayed rewards is a great challenge, since the consequences of an action are not immediately evident. Agents must learn to make decisions that may not yield immediate benefits, but lead to higher cumulative rewards in the future [3]. This involves finding a balance between immediate rewards and future goals, a process that can be difficult due to the temporal gap between actions and their eventual results [44].

To formalise this notion, we define the return G_t as the total cumulative reward obtained from the time step t onwards.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t} R_T \quad (2.5)$$

Here, R_t represents the reward obtained at time t , while γ is the discount factor that controls how much future rewards contribute to the present value and will retain this definition throughout the remainder of the chapter. T denotes the final time step in episodic tasks. The main objective in [RL](#) is to maximise the expected return, which is represented by the value function [45]. This expected return is mathematically expressed as:

$$\mathbb{E}[G_t] = \mathbb{E}[R_{t+1} + R_{t+2} + \cdots + R_T \mid S_t = s] \quad (2.6)$$

In cases where the problem involves a continuous task rather than an episodic one, i.e., when T equals infinity, γ is introduced to prevent infinite accumulation of rewards, leading to:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.7)$$

This formulation allows [RL](#) agents to learn optimal policies even in infinite-horizon tasks [46]. The Bellman equation builds on this definition, recursively decomposing value functions to enable practical learning strategies in [RL](#), as discussed in more detail in Section 2.4.2. [3, 45]

2.4.1 Markov Decision Process

RL is based on the **Markov Decision Process (MDP)** [46, 47] framework, which serves as a mathematical foundation for modelling decision-making in changing environments. In RL, an agent interacts with its environment by observing states, selecting actions, and receiving rewards, which together represent the fundamental components of an MDP. The MDP framework establishes the principles that dictate state transitions and reward systems, allowing the agent to develop optimal policies through interaction and learning from experience. Since RL algorithms are based on estimating and optimising value functions over time, MDP serves as the underlying foundation that allows these models to formalise decision-making, making them essential for solving sequential tasks. This structured approach enables RL agents to make informed decisions by maximising expected cumulative rewards while taking into account uncertainties in the environment. [3, 46]

Expanding on this foundation, an MDP builds on the Markov chain by introducing actions and rewards, making it a valuable tool for handling sequential decision-making in uncertain environments. An MDP represents the interaction between an agent and its environment, where each action affects the future states and the total rewards collected over time, as shown in Figure 2.4. This mathematical model enables the development of optimal policies that guide agents towards maximising long-term rewards while efficiently handling probabilistic state transitions and stochastic outcomes [3, 46].

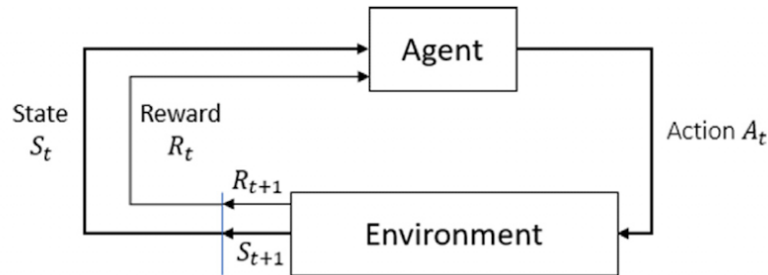


FIGURE 2.4: Agent-Environment interaction in an MDP (adopted from [3])

2.4.1.1 Markov Chains

A **Markov chain** is a basic stochastic process that follows the Markov property, meaning the likelihood of moving to the next state is determined only by the current state and not by previous states [48]. The Markov chain is made up of

- a finite set of states S , and

- a transition probability matrix P , where each entry $P(s'|s)$ represents the probability of moving from state s to state s' .

This process is fully described by the transition dynamics and is used in probabilistic modelling [46].

2.4.1.2 Markov Reward Process

The Markov reward process improves the Markov chain by introducing a reward structure. It builds on states S and the transition probability function $P(s'|s)$ defined in Section 2.4.1.1 and further includes the reward function which determines the immediate reward associated with a particular state or a transition between states. It can be expressed as $R(s)$, which represents the expected reward for being in state s , or more generally as $R(s, a, s')$, which defines the reward received for taking action a in state s and moving to state s' .

The objective in a Markov reward process is to compute the state-value function $V^\pi(s)$, which represents the expected return from a given state,

$$V^\pi(s) = \mathbb{E}_\pi [R_t^Y \mid S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{T-t} \gamma^k R_{t+k} \mid S_t = s \right] \quad (2.8)$$

where γ ensures that the agent prioritizes immediate gains while still considering long-term consequences [49].

2.4.1.3 Markov Decision Process

Expanding on Markov reward process, **MDP** incorporates actions that allow an agent to interact with its environment to achieve specific goals. An **MDP** is formally represented as a tuple (S, A, P, R, γ) , where S refers to the set of all possible states the agent can experience and A represents the set of actions the agent can choose from.

In an **MDP**, the objective is to develop an optimal policy $\pi(a|s)$ that determines the best possible action to take in a given state to maximise the total expected rewards over time [41]. To evaluate policies and estimate long-term action returns, **RL** is based on value functions, which will be explored further in Section 2.4.2.

2.4.1.4 Exploration vs. Exploitation

A major challenge in **MDP** is finding the right balance between exploration, which involves testing new actions to gain more information, and exploitation, which focusses

on selecting the best known action to maximise rewards. Several strategies help address this trade-off, including

- the **epsilon-greedy** method, where the agent sometimes chooses a random action with a chance of ϵ and at other times chooses the best known action with a chance of $1 - \epsilon$. This helps the agent explore new options while still using what it has learned [41, 50], and
- **entropy regularisation** that encourages exploration by incorporating an entropy term to the objective function which prevents the agent from settling too quickly on a suboptimal policy [38].

2.4.2 Bellman Equation

Building on the foundational concepts of **MDP**, the **Bellman equation** plays a vital role in **RL** by providing recursive decompositions of value functions. These equations show that the value of a state is calculated by adding the reward received immediately to the expected value of future rewards, which are discounted to account for their distance in time, allowing sequential decision problems to be broken down into smaller and more manageable parts [41]. For a given policy π , $V^\pi(s)$ satisfies the Bellman equation:

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) \sum_{s' \in S} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')] \quad (2.9)$$

This equation determines the expected return by accounting for all possible actions and future states, assigning weights based on their likelihood under the given policy π [3].

Similarly, the **action-value function** $Q^\pi(s, a)$, which evaluates the value of taking action a in state s under policy π , is defined as:

$$Q^\pi(s, a) = \sum_{s' \in S} P(s'|s, a) \left[R(s, a, s') + \gamma \sum_{a' \in A} \pi(a'|s') Q^\pi(s', a') \right] \quad (2.10)$$

Additionally, $V^\pi(s)$ can be written using $Q^\pi(s, a)$ as follows:

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) Q^\pi(s, a) \quad (2.11)$$

This relationship illustrates that the expected value of being in a state can be calculated as the weighted sum of action-values, where the weights correspond to the probability distribution of the policy over actions.

Similarly, the ideal value function and action-value functions are written as follows:

$$V^*(s) = \max_{\pi} V^{\pi}(s) = \max_{a \in A} Q^*(s, a) \quad (2.12)$$

$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a) \quad (2.13)$$

The recursive formulas in equations 2.12 and 2.13, referred to as Bellman optimality equations, are important because they link the value of a state (or a state-action pair) to the values of the states that follow. This relationship allows iterative techniques such as the value iteration and policy iteration to be used to solve MDP and find optimal policies. Using these recursive relationships, the RL algorithms can efficiently compute optimal policies that maximise the expected cumulative rewards [3, 46, 49].

2.4.3 Reinforcement Learning Algorithms

RL algorithms aim to optimise decision making in MDP without requiring explicit transition models [46]. Unlike Dynamic Programming, which assumes full knowledge of the environment, RL learns through direct interaction, overcoming challenges such as the Curse of Dimensionality and Curse of Modelling [51]. Most RL algorithms rely on the Bellman equation and the Generalized Policy Iteration (GPI), a step-by-step process that switches between policy evaluation, which estimates how good a given policy is, and policy improvement, which adjusts the policy to increase expected rewards [12]. This framework ensures continuous refinement of the policy until it converges to an optimal solution. The main RL algorithm categories, which differ in how they estimate values and optimise policies, are the following;

Monte Carlo methods estimate value functions by averaging returns over full episodes without using Bellman equations. They are particularly useful for policy evaluation and hyper-parameter tuning but require on-policy learning, meaning the same policy must be used for data collection and evaluation. This makes them inefficient for real-time learning, especially when the policy changes frequently. [42]

Temporal Difference (TD) learning improves on Monte Carlo methods by updating value estimates at each step rather than waiting for full episodes. This approach combines the benefits of Monte Carlo sampling and Dynamic Programming, making it more sample

efficient. However, since TD learning estimates values based on other estimates rather than complete returns, it introduces bias to its learning. [42]

Policy Gradient (PG) methods take a different approach by directly optimising policies rather than learning value functions. They are particularly useful in continuous and high-dimensional action spaces, where the synthesis of an optimal policy from a value function is infeasible [41]. Additionally, PG methods naturally explore, due to their stochastic nature, and are well suited for partially observable MDPs. However, they suffer from high variance and slow convergence, often requiring techniques such as baseline subtraction to stabilise learning. [42]

2.4.3.1 Function Approximation Techniques in Reinforcement Learning

Function approximation techniques are fundamental to RL, allowing agents to operate in large or continuous state and action spaces where tabular methods become impractical. MLPs form a fundamental architecture in RL, capable of approximating non-linear functions in both state and action spaces. Their fully connected layers allow for flexible feature extraction, making them a widely used function approximators. Due to their ability to model non-linear relationships, MLPs are particularly useful in value-based and policy-based methods, which will be discussed in more detail in Sections 2.5.3 and 2.5.4.

CNNs are commonly used in RL tasks that involve spatial data, such as image-based environments. By applying convolutional layers, CNNs can extract hierarchical features from raw input, which is valuable in environments where recognising spatial patterns is important for decision making.

RNNs are particularly useful in RL tasks where decisions depend not only on the current state but also on past observations. Traditional RL models are often based on MDP, which assume that the current state contains all the necessary information for decision making. However, in many real-world scenarios, environments are partially observable, which requires agents to infer hidden states by learning from past experiences. RNNs address this challenge by maintaining memory across time steps, allowing agents to capture temporal dependencies in sequential data.

2.5 Deep Reinforcement Learning

Building upon the foundational principles of MDP and the Bellman equations, DRL extends classical RL by incorporating DNNs to approximate value functions and policies. Traditional RL methods, such as value iteration and policy iteration, rely on tabular representations of value functions, which become computationally infeasible in state and

action spaces with a large number of variables or features. **DRL** addresses this challenge by using function approximation techniques, particularly **DNNs**, to handle high-dimensional and continuous environments where classical methods fall short [3, 46, 49].

RL models can be broadly categorised into three main approaches:

- **Model-Based RL** which constructs an explicit environment model, enabling planning and simulation, but often requiring extensive domain knowledge and computational complexity,
- **Model-Free Value-Based RL** which learns from past experiences without requiring a strict action trajectory. Algorithms such as Q-learning [18] and **Deep Q-Networks (DQNs)** [18, 52] fall under this category and perform well in discrete action spaces but struggle in continuous control tasks, and
- **Model-Free Policy-Based RL** which continuously refine a single policy throughout training, employing policy gradient methods, which are particularly useful for continuous action spaces.

The categorisation of the different RL approaches and their corresponding algorithms is illustrated in Figure 2.5. While many more algorithms exist beyond those shown, the diagram highlights the most relevant ones for the scope of this thesis. The remainder of this chapter provides a detailed examination of each of these categories.

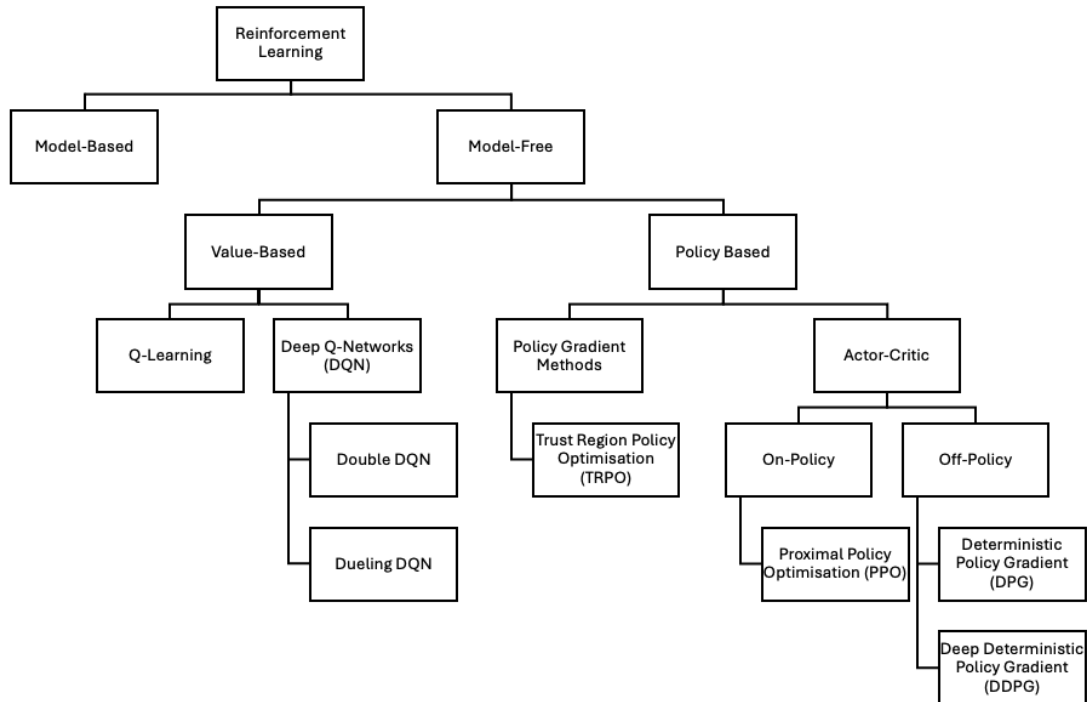


FIGURE 2.5: Tree diagram of different RL algorithm categories.

2.5.1 Model-Based Reinforcement Learning

Model-based RL is based on an internal model of the environment to predict the upcoming states and rewards, allowing the agent to plan its actions in advance before carrying them out. This approach improves learning by reducing the need for direct interaction with the environment. Although model-based RL can converge more quickly using prior knowledge, it also introduces challenges such as increased computational demands and potential inaccuracies in the learnt model, which can negatively affect overall performance. These limitations make model-based RL more suitable for structured environments, such as robotics and simulations, where a reliable model can be constructed [3]. A popular model-based algorithm is Dyna-Q [3], which integrates model-free Q-learning [53] with an internal model and is used in strategic decision making tasks [43].

2.5.2 Model-Free Reinforcement Learning

Model-free RL, as opposed to model-based techniques, learns optimal policies directly without building an explicit model of the environment. Rather, the agent engages with the environment and modifies its policy in response to perceived transitions and rewards [3]. This approach is frequently employed in high-dimensional, dynamic contexts where it is impossible to define an exact model.

Although model-free RL offers greater flexibility and simpler implementation, it is generally less sample efficient and requires extensive trial-and-error learning. Furthermore, it suffers from slower convergence and training instability, particularly in DL-based implementations [50]. Despite these challenges, model-free RL remains a practical approach in applications such as financial trading and robotic control, where precise modelling of the environment is impractical. As mentioned in Section 2.5, model-free RL can be further divided into two primary categories:

- **Value-Based methods**, which are methods that learn from previously collected experiences rather than rely solely on real-time interactions. Value-policy methods enable agents to reuse past data, improving sample efficiency. A prominent example is Q-Learning, where the agent updates the action value function $Q(s, a)$ based on stored experiences rather than requiring immediate feedback. DQN extend this by incorporating DNNs for function approximation, improving performance in challenging environments [50], and
- **Policy Based methods**, which are methods that continuously improve the policy while interacting with the environment. Unlike off-policy methods, on-policy

learning updates the policy based on the most recent trajectory, making it more stable but often requiring more data. Policy gradient methods and PPO optimise a policy $\pi(a|s)$ directly to maximise the expected cumulative rewards, which is particularly advantageous in continuous action spaces [3, 38].

Furthermore, actor-critic methods combine aspects of both approaches by learning a policy while simultaneously evaluating its performance using a value function. These hybrid methods aim to mitigate the weaknesses of purely value-based or policy-based techniques, offering improved learning stability. [54]

2.5.3 Value-Based Methods

Value-based methods are centred on learning the value of actions to guide decision making. These approaches work particularly well in discrete action spaces, where an agent must choose from a fixed number of possible actions. Instead of learning the policy directly, the agent estimates how good each action is in a given state using what is called an action value function.

The primary objective is to estimate the optimal Q-function $Q^*(s, a)$, which reflects the highest expected return that can be achieved by taking action a in state s , and continuing to act optimally [3, 46]. A major strength of this method is its reliance on the Bellman equation, which enables the model to refine these value estimates step by step based on new experiences.

This structure is the backbone **Q-Learning** and **DQN** [41, 55, 56], both of which will be covered in detail in Section 2.5.3.1. Although these methods have shown promising results, they also come with limitations;

- limited scalability to continuous action spaces, as they rely on maximising over discrete actions, making them ineffective for high-dimensional, continuous control problems,
- training instability, as deep Q-learning algorithms are sensitive to hyper-parameters and may suffer from divergence, and
- exploration inefficiencies, since methods like ϵ -greedy exploration often fail to sufficiently explore challenging environments.

To address these limitations, approaches such as Double **DQN** [18, 55], which reduces overestimation bias by using separate networks for action selection and Duelling **DQN** [56], which separates state-value and advantage estimation to improve learning efficiency, have been proposed.

2.5.3.1 Q-Learning

Q-Learning is a fundamental [RL](#) algorithm based on model-free value-based models that are designed to find the optimal $Q^*(s, a)$ without the need for prior information on how the environment works. The main concept behind Q-Learning is to repeatedly adjust Q-values using the Bellman equation, allowing the agent to estimate the expected total reward for each state-action pair through experience, as seen in the following:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (2.14)$$

where:

- α represents the learning rate which determines how much influence new information has on updates,
- R denotes the reward obtained immediately after performing action a , and
- $\max_{a'} Q(s', a')$ represents the highest expected reward in the next state s' to guide the update process.

Q-Learning follows an off-policy learning approach, which means that it updates the Q-values by considering the greedy policy, even if actions are selected using an exploration strategy like ϵ -greedy [3]. This ensures that the algorithm balances exploration and exploitation. Q-learning performs well in discrete environments but struggles with high-dimensional state spaces, which requires the development of function approximation techniques such as [DQN](#), which address these limitations by integrating [DL](#) methodologies [46, 49].

2.5.3.2 Deep Q-Learning

Deep Q-Learning is an improved version of Q-Learning that uses [DNNs](#) to estimate Q-values, helping the agent learn in large environments. Instead of storing all possible values, [DNNs](#) helps guess the best actions, making it easier to handle large state spaces [3].

The goal of training a [DQN](#) is to reduce the discrepancy between the target Q-value and the current Q-value estimate, which is determined by applying the Bellman equation as follows:

$$Q(s, a; \theta) \leftarrow Q(s, a; \theta) + \alpha \left[R + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right] \quad (2.15)$$

where:

- θ represents the set of weights in the Q-network, and
- θ^- refers to the target network, which is updated at intervals to keep training stable.

Experience Replay is a crucial technique introduced in [DQN](#) to enhance training stability. Instead of updating the network with the most recent transition, experiences are stored in a replay buffer and randomly sampled to break the correlation between consecutive updates, improving convergence and sample efficiency [\[49\]](#).

Another major improvement in [DQN](#) is the use of θ^- , which is updated less frequently than the main Q-network. This mitigates the instability caused by directly bootstrapping Q-values from a constantly changing network [\[46\]](#).

However, despite these improvements, value-based methods remain fundamentally limited in continuous action spaces. So to overcome these challenges, Policy-Based Methods have been developed, offering an alternative approach that directly learns optimal policies without relying on action-value estimation.

2.5.4 Policy-Based Methods

Unlike value-based methods, which estimate the value of actions to decide what to do, policy-based methods focus directly on learning a policy that maps each state to an action, without needing a separate value function or critic. Rather than picking actions by maximising a Q-value, these methods train a parametrised policy $\pi(a|s; \theta)$ that gives a probability distribution over actions for a given state. The goal is to increase the total reward that the agent can expect by tweaking the policy parameters through gradient-based optimisation [\[3, 38\]](#).

2.5.4.1 Policy Gradient Methods

Policy gradient methods are a popular way to implement this idea. They work by shifting the policy parameters in a direction that is expected to increase rewards over time. In contrast to value-based approaches, which estimate Q-values, policy gradient methods focus directly on learning and improving the policy function $\pi(a|s; \theta)$, often using stochastic gradient descent [\[57\]](#).

These methods update the policy by estimating how the expected return would change with respect to the current policy parameters. The general form of this update is as follows.

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} \left[A_t \cdot \nabla_{\theta} \log \pi(a|s; \theta) \right] \quad (2.16)$$

Here, A_t is an advantage function, which helps reduce the variability in the gradient estimate, making learning more stable. However, policy gradient methods are known to suffer from high variance and may require a large number of samples to achieve reliable learning. This makes sample efficiency a key concern. To mitigate this issue, methods such as [Trust Region Policy Optimisation \(TRPO\)](#) [3] introduce constraints on policy updates to prevent large, destabilising shifts between iterations, thus improving training stability [38, 41]. Despite these limitations, policy gradient methods remain central to [RL](#), especially in settings that involve continuous or stochastic action spaces [58, 59].

2.5.4.2 Policy-Based Methods vs Value-Based Methods

Policy-based methods present several advantages over their value-based counterparts, particularly in environments characterised by continuous action spaces. Unlike value-based approaches, which rely on discrete action selection and are constrained by the max operator, policy-based methods can learn a direct mapping from states to actions, facilitating smoother adaptation to high-dimensional control tasks. They also exhibit improved convergence properties, as the absence of bootstrapped maximisation reduces the risk of instability and overestimation bias. Furthermore, the use of stochastic policies inherently encourages more diverse exploration, thereby mitigating the risk of premature convergence to suboptimal behaviours. However, policy gradient methods are not without limitations; they often suffer from high variance in gradient estimates and require large volumes of data to learn effectively. Actor-critic architectures serve to address these challenges by incorporating a value function to guide policy updates, combining the strengths of both paradigms and rendering policy-based methods applicable to a broader range of reinforcement learning problems. [38]

2.5.5 Actor-Critic Methods

To address the high variance in policy gradient methods, actor-critic methods introduce a separate value function (critic) that helps guide the policy update. The actor updates the policy, while the critic evaluates the expected return of a given state-action pair [38, 46], reducing the variance in the policy gradient estimate, as seen in [Figure 2.6](#).

The advantage function $A(s, a)$ is used to refine the update rule by allowing the policy update to focus more on actions that perform better than expected [3], given by:

$$A(s, a) = Q(s, a) - V(s) \quad (2.17)$$

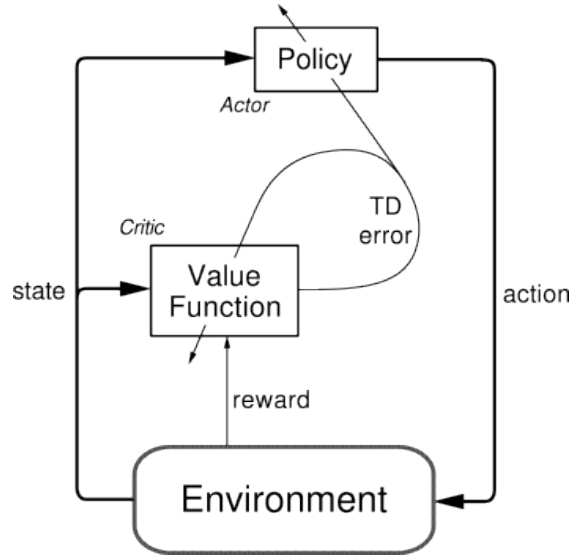


FIGURE 2.6: The actor-critic architecture (adopted from [3])

Actor-critic methods can be further categorised into on-policy and off-policy approaches. On-policy algorithms, such as A2C and PPO, learn using data gathered from the current policy, which tends to support more stable learning. In contrast, off-policy methods, such as [Deterministic Policy Gradient \(DPG\)](#) and [Deep Deterministic Policy Gradient \(DDPG\)](#) learn from data generated by a different policy, allowing greater flexibility in using past experiences, particularly in continuous action settings.

2.5.5.1 Deterministic Policy Gradient

While traditional policy gradient methods learn $\pi(a|s)$, [DPG](#) methods directly optimise a deterministic policy $\mu_\theta(s)$, which outputs a specific action rather than a probability distribution over actions [60].

The policy gradient update in [DPG](#) is given by,

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim \rho^\pi} \left[\nabla_\theta \mu_\theta(s) \cdot \nabla_a Q^\mu(s, a) \Big|_{a=\mu_\theta(s)} \right] \quad (2.18)$$

where:

- $Q^\mu(s, a)$ is the action-value function under policy μ , and
- ρ^π is the state distribution under policy π .

[DPG](#) enables direct optimisation of continuous actions without the need to sample a probability distribution, making it particularly suitable for high-dimensional continuous control tasks in financial trading [41].

2.5.5.2 Deep Deterministic Policy Gradient

DDPG [18, 52] methods extend **DPG** by incorporating the replay experience and the target networks, similar to **DQN**, to improve stability and learning efficiency [52]. **DDPG** maintains

- an **actor network** which learns $\mu_\theta(s)$, and
- a **critic network**, which estimates $Q(s, a)$ using the Bellman equation.

The use of target networks prevents divergence during training, while experience replay helps de-correlate consecutive samples, leading to more stable learning.

2.5.5.3 Proximal Policy Optimisation

PPO is a **RL** algorithm designed to improve training stability. Instead of making drastic policy changes that can lead to instability, **PPO** introduces a clipping mechanism to limit how much the policy is allowed to change during each update. This helps prevent the model from deviating too far from what it already knows, while still allowing it to improve.

The idea is to strike a better balance between exploring new strategies and sticking to what already works. By softly restricting how far the new policy can drift from the old one, **PPO** ensures that the updates stay within a reasonable range. This approach helps the algorithm learn more steadily over time.

The main **PPO** loss function, shown in Equation 2.19, uses a clipped objective that compares the current policy with the previous to limit excessive updates. This is done using the probability ratio $P_t(\theta)$, which compares the likelihood that the current policy will take an action versus the old policy as seen in Equation 4.5, and applying a clipping threshold ϵ to prevent large deviations. This balance helps the algorithm remain stable while still allowing meaningful improvements to the policy [38]. The **PPO** objective is defined as:

$$L_{\text{CLIP}}(\theta) = \mathbb{E}_t [\min (P_t(\theta)A_t, \text{clip}(P_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (2.19)$$

By clipping policy updates, **PPO** stabilises training and achieves better sample efficiency, making it one of the most used policy optimisation methods in **DRL**. [38, 61, 62]

2.5.6 Special Cases of Actor-Critic Methods

Although actor-critic methods integrate both value-based and policy-based techniques, it is useful to examine their special cases, actor-only and critic-only methods. These approaches rely exclusively on either policy learning or value estimation, each with distinct advantages and limitations, as seen in Sections 2.5.6.1 and 2.5.6.2.

2.5.6.1 Actor-Only Method

Actor-only methods are [RL](#) approaches that rely solely on learning a policy without using a value function. These methods directly optimise $\pi(a|s; \theta)$ to maximise the expected cumulative reward, often using policy gradient techniques. Since they do not estimate value functions, actor-only methods work well in continuous action spaces and scenarios where learning an explicit value function is challenging [3, 60].

However, in complex environments, convergence is slow and unstable because of the significant variance in gradient estimations of the actor-only approach. To address these limitations, entropy regularisation is often introduced to encourage exploration, preventing premature convergence to suboptimal policies [63].

2.5.6.2 Critic-Only Method

Critic-only methods are [RL](#) techniques that focus solely on learning value functions, without explicitly learning a policy. These methods optimise decision making by using the estimated value function to guide action selection, typically using a greedy or heuristic-based policy [3, 46].

Critic-only approaches work well in discrete action spaces where explicit policy representation is unnecessary. However, they struggle in continuous action settings, as selecting actions requires solving an optimisation problem at each step [64, 65].

To mitigate this issue, some critic-only methods incorporate neural networks, to learn value functions more efficiently. As seen in Section 2.5.3.2 a notable example is [DQN](#), which uses a neural network to approximate $Q(s, a)$ and applies experience replay to stabilise training [50].

Although critic-only methods excel in environments where value estimation is more efficient than policy optimisation, they are often complemented by actor-based approaches, leading to actor-critic architectures that balance the strengths of both paradigms.

2.6 Conclusion

This chapter reviewed the progression from traditional [ML](#) to [DRL](#), while focussing on neural network architectures and their roles in function approximation. It also introduced [RL](#) concepts, including [MDPs](#), value functions, and modern algorithmic approaches. Chapter [3](#) examines the data used in this study, outlines its structure and relevance, and provides the foundation for the methodology presented in Chapter [4](#).

Chapter 3

Data

In modern data science, the vast availability of data has transformed how the world analyses and interprets large-scale systems. Almost every interaction, transaction, or event now leaves behind a digital footprint. This continuous flow of information creates new opportunities for analysis and insight, particularly in fast-paced environments such as financial markets, where data is produced rapidly and on a large scale.

One helpful way to think about the nature of data in the world today is through the concept of the "three V's": volume, velocity, and variety. These dimensions capture some of the key challenges and characteristics of working with big data. Volume describes the large quantity of data that is generated, velocity relates to how quickly this data is created and must be handled, and variety captures the wide range of formats and types that the data can take, from numerical price data to categorical indicators, and even textual sources such as news or social media sentiment.

The rest of this chapter walks through the dataset used in this project, outlines its structure, and explains why it is relevant to the analysis carried out in this project.

3.1 Data Source Selection

There are numerous options to access stock market data, ranging from free public [Application Programming Interfaces \(APIs\)](#) to commercial data platforms that provide institutional-level services. Premium providers such as Bloomberg Terminal [66], TickData [67], and Refinitiv [68] offer fine-resolution market data with comprehensive historical coverage that can span over 30 years. These datasets are usually cleaned and adjusted for corporate actions, such as dividends and stock splits, and often include extended trading session data. However, access to these platforms involves substantial

subscription costs and strict licencing terms, making them impractical for most academic research projects.

For this project, a free and open access solution was found. Yahoo Finance [69] was chosen as the main data source due to its convenience, availability, and broad coverage of financial indicators. The Yahoo Finance [API](#) offers historical stock data in daily, weekly and monthly resolutions. It supports intra-day intervals as fine as one minute; however, access to these high-resolution data is typically limited to the most recent 60 days. For longer time periods, [API](#) provides hourly data for up to three years, as well as extended daily series, depending on the historical coverage of the specific stock.

However, Yahoo Finance is not without limitations. One key issue is the inconsistency in historical coverage between different tickers, where older data is sometimes incomplete or missing entirely, especially for less liquid assets [70]. Moreover, the platform has occasionally returned unstable or missing data for certain time periods, as noted by users [71]. For this reason, the dataset used in this project was carefully cross-checked for gaps and irregularities as discussed in Section 3.4. Ultimately, data from 2020 onwards were used for analysis, to avoid training or testing models under older market conditions that may not align with current dynamics.

An alternative source that was briefly considered was Alpha Vantage [72], which offers time-series data and a rich set of technical indicators via its [API](#). Although Alpha Vantage provides a user-friendly interface and a generous free tier, it enforces strict request limits that slow down data acquisition for larger datasets. Its documentation is also more limited, and some indicators are only available with a paid key. Despite these drawbacks, Alpha Vantage could be a valuable resource to supplementing price data or generating engineered features in future studies.

3.1.1 Data Acquisition

The dataset was obtained from Yahoo Finance using the open-source Python library *yfinance*, which offers a programmatic interface for accessing historical market data via the platform's [API](#). As discussed in Section 3.1, this [API](#) allows the specification of ticker symbols, time intervals, and date ranges directly within Python scripts, making it suitable for efficient data extraction.

Although the [API](#) supports hourly and even minute-level data, this project intentionally focused on daily intervals. Using daily data reduces the noise of short-term price fluctuations while still capturing the broader trends that are most relevant for medium- and long-term trading strategies. It also eases computational demands, offering a more

tractable solution for model training, especially when working with multiple stocks over several years.

To keep the workflow modular and manageable, each ticker's data was saved as an individual [comma-separated values \(CSV\)](#) file. This format was chosen for its simplicity and compatibility with most data processing tools. Within each file, as seen in Table 3.1, the first column captured the 'Date' and the second recorded the 'Closing Price', which served as the main reference for the analysis.

TABLE 3.1: Sample structure of downloaded stock data using daily intervals

Date	Closing Price
2022-01-03	294.42
2022-01-04	289.12
2022-01-05	281.78
2022-01-06	283.55
2022-01-07	289.10
...	...

3.2 Security Selection

In financial modelling, selecting the appropriate set of securities is more than just a preliminary step. Different stocks behave differently depending on the industry they belong to, as some exhibit higher volatility, others are more stable, and all respond uniquely to market conditions [73]. Since [RL](#) models need to adapt to these patterns, the selection of securities impacts not only the results, but also the way the model is tuned and trained.

Table 3.2 summarises the eleven Global Industry Classification Standards currently used in financial markets, together with their respective average net profit margins.

TABLE 3.2: Overview of the eleven Stock Market Sectors

Sector	Common Industries	Avg. Net Profit Margin (%)
Communication Services	Telecoms, media, entertainment, interactive services	12–15%
Consumer Discretionary	Retail, apparel, automotive, leisure products	6–10%
Consumer Staples	Food, beverages, household goods, personal care	10–13%
Energy	Oil, gas, renewable energy, refining	7–12%
Financials	Banks, insurance, investment services	15–20%
Health Care	Pharmaceuticals, biotech, healthcare services	10–17%
Industrials	Aerospace, defence, manufacturing, logistics	6–9%
Technology	Software, hardware, semiconductors, IT services	18–25%
Materials	Chemicals, metals, construction materials	5–9%
Real Estate	Property management, real estate development	10–14%
Utilities	Electric, gas, water utilities	8–10%

To build a reliable and informative dataset, it was important to choose stocks with data that was not only consistent and widely available, but also showed enough variation over time for the model to learn from. Initially, the project considered training models across multiple sectors of the stock market to capture a wider range of market behaviours and dynamics. However, to ensure a more controlled experimental setup and facilitate a clearer interpretation of results, the scope was refined to a single sector. The technology sector was selected for this purpose.

Technology stocks generally offer high trading volume and liquidity, which means that they provide frequent price movements, thus ideal for [RL](#) models that rely on ongoing feedback from the market. Companies such as Apple and Microsoft benefit from clean and complete historical data, reducing the risk of gaps or inaccuracies that could interfere with training. The technology sector has also been at the centre of major market trends and innovation in recent years, offering a rich and realistic context for learning. In addition, many leading technology firms are heavily weighted in widely used market indices such as the NASDAQ-100 and S&P 500 [69], which makes them useful for comparison with benchmarks and existing studies. Compared to sectors like energy or finance, technology is generally less affected by regulatory disruptions or policy cycles, allowing the model to focus more on learning from the market itself rather than external events.

3.2.1 Overview of Selected Stocks

Following the decision to focus on the Technology sector, a set of large-cap, high-liquidity companies was shortlisted to include in the dataset as seen in Table 3.3.

TABLE 3.3: Summary of Selected Technology Stocks

Ticker	Company	Primary Industry
AAPL	Apple Inc.	Consumer Electronics
MSFT	Microsoft Corp.	Enterprise Software/Cloud Computing
NVDA	NVIDIA Corp.	Semiconductors/AI Hardware
GOOGL	Alphabet Inc.	Digital Advertising/Search Engines
AMZN	Amazon.com Inc.	E-commerce/Cloud Infrastructure
TSLA	Tesla Inc.	Electric Vehicles/AI/Automation
META	Meta Platforms Inc.	Social Media/VR Platforms

These assets were chosen because of their availability of clean and uninterrupted historical data. The *yfinance* API occasionally returns incomplete records, especially for low-volume or thin-traded stocks. Stocks such as BlackBerry Ltd. (BB) and GoPro Inc. (GPRO) were initially considered, but ultimately excluded due to erratic price behaviour, extended flat trading periods, and inconsistent volume. These anomalies, often linked to low liquidity or speculative trading, risk introducing structural noise into the learning process, which confuses the agent during training and undermines model reliability.

The retained stocks not only have consistently high trading volumes and complete historical records, but also exhibit a wide range of volatility profiles, as shown in Figure 3.1. For example, Tesla and Nvidia display broader return distributions, signalling more frequent and larger price swings, while Google and Microsoft are comparatively more stable. This variation is valuable for RL, as it exposes the agent to both high- and low-risk market conditions.

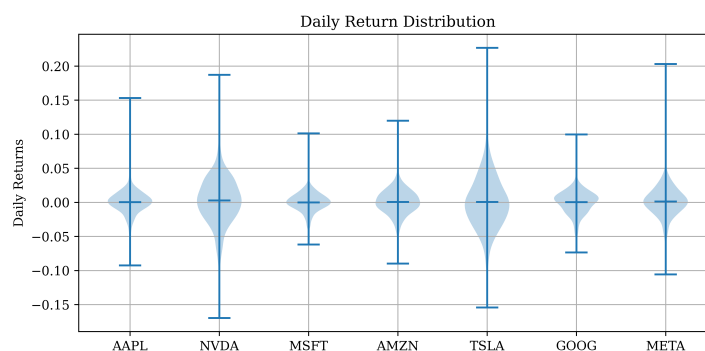


FIGURE 3.1: Violin plot comparing the daily return distributions for the selected stocks.

Furthermore, the diversity of business models represented, such as Amazon’s sensitivity to seasonal retail sales and Amazon Web Services growth, or META’s reliance on digital advertising, adds further depth to the dataset. These differences introduce a broader range of price behaviours and external dependencies, helping the model adapt to a variety of real-world market patterns.

3.2.2 Corporate Actions Adjustments

When working with historical market data, corporate actions such as stock splits, dividends, and mergers must be taken into account, as they can distort raw price trends and confuse pattern recognition in learning algorithms. Two notable events during the selected data range include the Apple 4-for-1 and Tesla 5-for-1 stock splits in August 2020. Although these actions did not change the overall valuations of the companies, they reduced the nominal share price, as seen in the left plot of Figure 3.2. Without adjustment, such shifts might appear as steep drops, misleading both visual inspection and algorithmic learning.

Another advantage of using the *yfinance* API is that it automatically applies price adjustments, accounting for corporate actions without requiring additional preprocessing. These adjustments help ensure continuity in the dataset by neutralising artificial changes that result from corporate actions rather than actual market behaviour, as seen in the right plot of Figure 3.2.

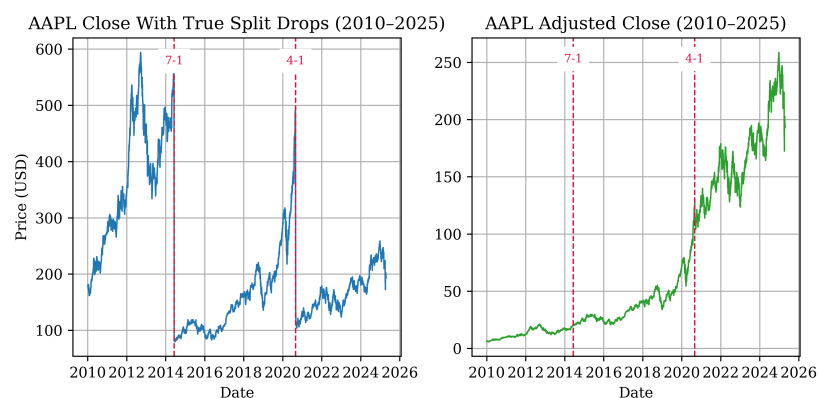


FIGURE 3.2: Effect of Apple Stock Split on Price Data: Unadjusted Closing Prices Showing Apparent Price Drop (Left) and Adjusted Closing Prices Reflecting True Continuity (Right)

3.2.3 Impact of Market Events

In addition to structural adjustments, broader market events also have a significant impact on the learning environment. As shown in Figure 3.3, the data set covers key market phases, including the COVID-19 crash in early 2020 (shaded in red) that brought intense volatility and unpredictable price swings. These conditions make it harder for RL agents to converge on stable policies, but they also create a valuable test basis to evaluate how well an agent can adapt under unforeseeable market conditions. In contrast, the *tech rally* [74] that followed (shaded in green) provided a long stretch of upward price movement and relatively steady growth. This offers a different type of learning, allowing the model to understand how to handle trend-following behaviour under more predictable conditions. By including both stable periods and times of high volatility, the agent has the opportunity to learn how markets behave under different conditions. All of these filtering and adjustment choices were made to keep the dataset realistic while avoiding any misleading signals that could have biased the model during training.

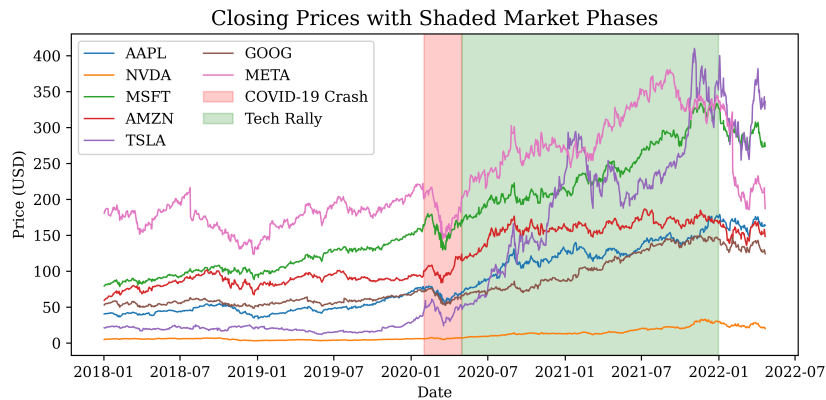


FIGURE 3.3: Closing prices of selected technology stocks from 2018 to 2022. The red region marks the COVID-19 crash (February 2020 to March 2020), and the green region indicates the *tech rally* (April 2020 to November 2021).

3.3 Market Indicators

3.3.1 Feature Engineering

Market indicators are mathematical tools derived from price and volume data that are widely used in trading to identify trends, momentum shifts, or overbought/oversold conditions. In the context of this project, several technical indicators were designed and included as part of the set of characteristics during exploratory tests. These features were used to investigate how much high-level structured information the neural network could need to make accurate predictions, as opposed to learning patterns solely from raw price data.

The technical indicators selected for this project were chosen not only for their popularity in the financial modelling literature but also for their complementary characteristics and individual contributions. Each indicator captures a distinct aspect of market behaviour, be it momentum, trend, or volatility, providing the model with diverse input during its stages. The aim was to assess how much structured guidance was needed for the agent to begin forming useful patterns. The selected indicators were:

- the [Exponential Moving Average \(EMA\)](#) [75] was calculated over a five day period to capture short-term price trends. Since it assigns more weight to recent prices, the [EMA](#) helps the model respond more quickly to changes in market direction compared to traditional moving averages,
- the [Relative Strength Index \(RSI\)](#) [76], was computed using a fourteen day window to serve as a momentum indicator. It is particularly useful for identifying when an asset might be overbought or oversold, which can signal a potential trend reversal or weakening momentum,
- [Moving Average Convergence Divergence \(MACD\)](#) [77], was calculated by taking the difference between the twelve day and twenty six day [EMA](#). This indicator helps the model detect shifts in momentum and trend direction by tracking the relationship between two moving averages and their signal line, and
- finally, **Bollinger bands** [78], which were calculated using a twenty day moving average along with the corresponding standard deviation to capture market volatility. The distance between the upper and lower bands changes based on how far recent prices move away from the average, hence widening when the market is more volatile and narrowing during calmer periods. This helps the model recognise

when the market is likely experiencing strong momentum versus times of sideways movement or consolidation.

3.3.2 Indicator Limitations

Although technical indicators provide useful summaries of market behaviour, they also have notable limitations. Most are based on historical data and, therefore, are inherently lagging. For example, moving averages tend to respond slowly to sudden price changes, which can delay decision making in fast-moving environments. In addition, indicators such as [RSI](#) or [MACD](#) can generate conflicting signals in sideways or noisy markets, making them less reliable during periods of low trend strength.

3.4 Data Preprocessing

Preprocessing was an essential part of this project, as the ability of the model to learn depends directly on the quality of the data it sees. If the input is noisy or inconsistent, the model may struggle to find patterns or make meaningful updates during training. This is especially true in [RL](#), where decisions are based on sequences of time-dependent input. Section [3.4.1](#) outlines how the data was cleaned, scaled, and structured to ensure that it was ready for training.

3.4.1 Data Integrity

The raw data was first cleaned to remove inconsistencies and ensure that the time series was continuous. Any rows with missing values (NaN) were dropped. These were generally scarce, often occurring on holidays or due to short-term trading suspensions, and their removal had no significant impact on coverage.

To prevent distorted learning due to errors or anomalies, outlier detection was applied using the modified z-score method. This approach flags points that significantly deviate from the median on the basis of the median absolute deviation. Although extreme price moves are a natural part of market dynamics, this step helped catch data irregularities, such as recording glitches or incomplete uploads that could mislead the model.

3.4.2 Data Normalisation

Normalisation is the process of scaling features so that they are on a similar range, which is important because raw stock prices may exhibit large variability. For example, as shown in Figure 3.3, NVDA trades at a much lower closing price than MSFT. If this difference is not adjusted, it can create a bias in the model since larger price changes in MSFT might dominate training, while smaller movements in NVDA could be ignored. Normalisation places all inputs on a comparable scale, allowing the model to identify patterns across assets without being biased by differences in magnitude.

Several normalisation techniques were explored to standardise the data, as seen in Figure 3.4, including:

- **Min-Max Scaling**, which rescales each feature to a $[0, 1]$ range,
- **Z-score Standardisation**, which adjusts the data so that the average value becomes zero and the spread of the data is scaled relative to its standard deviation, and
- **Final Price Normalisation**, where each value is divided by the final closing price in the series [79].

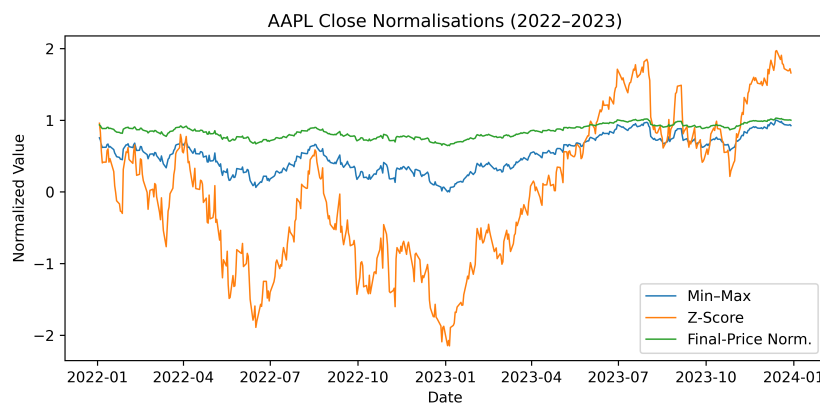


FIGURE 3.4: Visualisation of the different normalisation methods for Apple (2022-2023).

3.4.3 Final Data Format

The data for each stock were saved in a separate [CSV](#) file for modularity and cleaner processing. These files contained the cleaned features indexed by date. To prepare the data for training, a rolling window approach was used to construct input sequences. For any given time step t , the state was defined as a sequence of past k days;

$$s_t = [x_{t-k+1}, \dots, x_t]$$

The next state used for learning was constructed by shifting the window one day forward.

$$s_{t+1} = [x_{t-k+2}, \dots, x_{t+1}]$$

This method allowed the agent to learn from a stream of temporally ordered market conditions rather than from isolated data points. An example of a 3-day moving window using the downloaded closing prices and calculated features is shown in [Table 3.4](#).

TABLE 3.4: Example of 3-Day Rolling Window for calculated Features

Date	Closing_1	Closing_2	Closing_3	RSI_1	RSI_2	...
01-01-2022	78.4	74.9	74.7	49.4	43.0	...
02-01-2022	74.9	74.7	77.2	43.0	54.7	...
03-01-2022	74.7	77.2	77.8	54.7	57.6	...
04-01-2022	77.2	77.8	78.7	57.6	57.5	...
...	...	...	...	...	...	...

Each row represents one training sample, giving the model a short-term context on how price and trend signals evolve over time. When building these windows, the first few data points were skipped, since there are not enough previous days to fill a complete window. For example, the model cannot build a 3-day sequence starting from the first day because there is no data from two days before that. Removing these initial rows ensured that all training samples were properly structured and complete.

3.5 Buy-and-Hold Threshold

To evaluate whether models are learning to trade meaningfully, a simple baseline strategy was needed for comparison. In this project, the Buy-and-Hold approach was used as a benchmark. The logic is straightforward, at the start of the evaluation period, the total portfolio value is equally split across all selected stocks, and no further actions are taken until the end of the testing period.

Since it reflects how many investors approach the market, it works well as a reference point. If a model cannot outperform this passive strategy, it raises questions about whether it is actually learning useful patterns or just reacting to noise. Comparing the [RL](#) agent with this baseline helps determine whether it adds real value, especially in markets that are unpredictable or prone to sharp changes.

An initial investment of \$10,000 was split equally between the seven stocks and grew to \$17,379 during the 3-year training period, resulting in a total return of 73.8%, as illustrated in Figure 3.5. Although the portfolio experienced strong growth early on, a significant decline followed, particularly after September 2021. The buy-and-hold benchmark helps to clarify whether the agent can outperform a simple strategy that, while passive, has historically performed quite well, especially in bull markets.

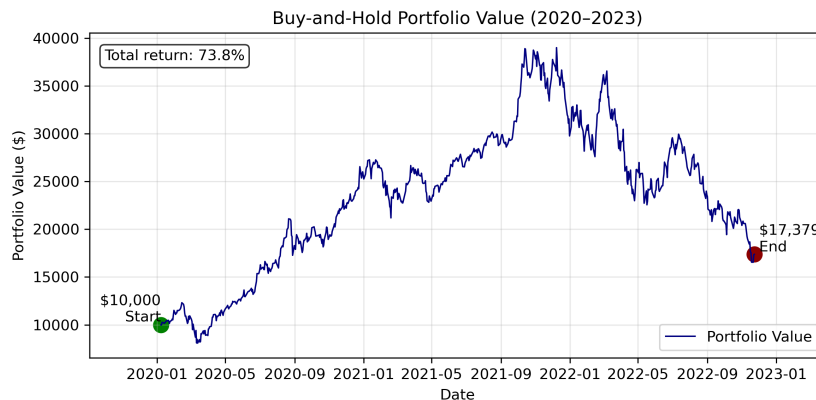


FIGURE 3.5: Buy-and-Hold portfolio growth.

3.6 Conclusion

This chapter laid the foundation for all subsequent modelling by detailing how the financial data was structured for the [ML](#) models to extract meaningful patterns. Each stage of preprocessing was needed to ensure statistical reliability and alignment with the requirements of [DL](#) frameworks adopted in this work.

Chapter 4

Methodology

This chapter explains the methodology used to develop and test the [ML](#) models for portfolio management. It covers both supervised and [RL](#) approaches, detailing how they were implemented, trained and evaluated based on the data presented in Chapter 3.

4.1 Software Infrastructure

All models in this project were developed using Python 3.12 which was chosen for its strong support and built-in functions needed in [DRL](#) implementations. Two popular libraries used for [DRL](#) are *TensorFlow* [80] and *Keras* [81], which offer tools for constructing and training DL models. Familiarity with these libraries was gained during the supervised learning implementation; however, when transitioning to [DRL](#), implementing custom training loops and manually managing gradients became complicated due to *TensorFlow*'s reliance on static computation graphs, which require the entire computation sequence to be predefined. This made debugging and tuning the [PPO](#) algorithm difficult. Therefore, *PyTorch* [82] was selected for the final implementations because it uses dynamic computation graphs, which means that the computations are defined and executed step-by-step at runtime. This flexibility made it easier to handle tensor operations, gradient updates, and weight adjustments. In addition, other Python libraries were used, with *NumPy* and *Pandas* for data processing, and *Matplotlib* for data visualisation and plotting results. The development was carried out using *Jupyter Notebooks* [83], which facilitated structured implementation and visual debugging.

The complete codebase is publicly available in this [GitHub Repository](#). The repository contains seven files, each of which includes a different model implemented. Every notebook is structured consistently, starting with data preparation and feature engineering, followed by defining and training the [RL](#) agent with [PPO](#), and integrating it within a

custom-built trading environment tailored for multi-stock continuous portfolio allocation. Each notebook includes functions for Monte Carlo evaluations to assess the model's performance across multiple environment runs.

The project followed an incremental development approach, starting with the implementation of a supervised learning framework to establish a baseline and identify informative market features that could be used as inputs to all other models. These insights allowed for easy integration and design of the entire [DRL](#) system.

4.1.1 Data Structure

Following the data preparation process described in Section [3.4.3](#), all models in this project were trained in the same data period, from 01-01-2020 to 01-01-2023. Longer training durations increase the exposure to outdated market conditions, potentially reducing the model's sensitivity to current patterns. This makes it more difficult for the agent to distinguish between persistent trends and transient noise, ultimately degrading model performance. A separate validation window, spanning from 01-01-2023 to 01-01-2024, was withheld entirely during development and used only for final evaluation, ensuring that this portion of the data remained completely unseen by the trained models. Figure [4.1](#) displays the raw closing prices for all stocks included in this study, with the training period shaded in yellow and the validation period in pink. These closing prices form the basis for all experiments, and the preprocessing steps vary depending on the learning approach being used.

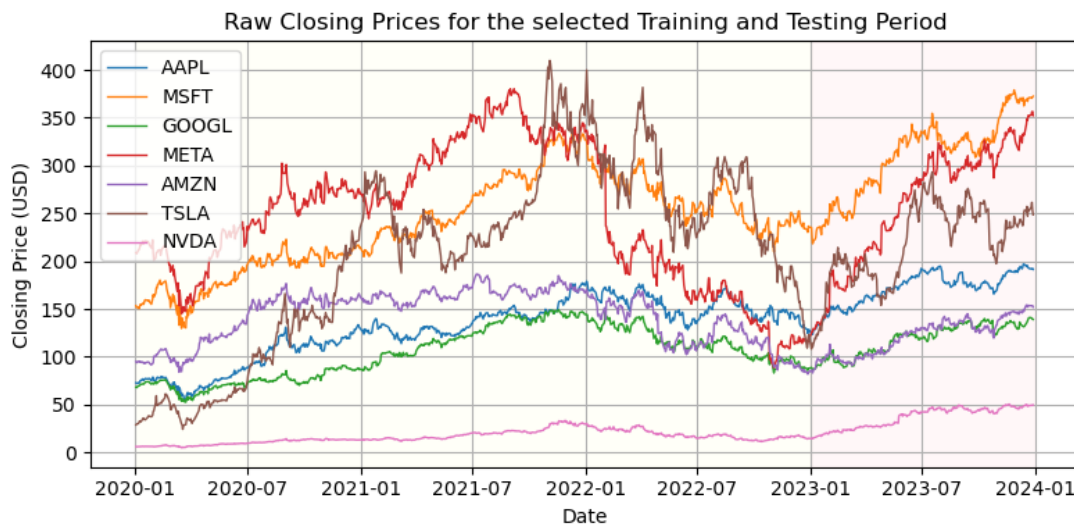


FIGURE 4.1: Raw closing prices for the selected stocks used in this study, with the training period shaded in yellow and the testing period shaded in pink.

4.2 Supervised Learning Implementation

To establish a foundation for the [RL](#) framework, an initial supervised learning model was developed using a [MLP](#) network. This model was trained on the raw closing prices and associated features described in [Section 3.3.1](#), using the pre-processing data outlined in [Section 3.4.3](#) to generate structured learning targets.

4.2.1 Data Preprocessing

A new column, labelled "Action", was added to the dataset to indicate whether the model should buy, hold, or sell a stock. Since supervised learning requires labelled targets, this column served as the label that the model would learn to predict. The first step in defining these labels was to calculate the daily price difference from the closing prices. Although the price difference does not give a complete picture of the market behaviour, it is a simple and intuitive measure of the directional movement of a stock. Instead of relying on arbitrary thresholds, a quantile-based approach was used to define the labels given by 'Sell', 'Hold', 'Buy' or 'Strong Buy' for each stock. A histogram was generated, as seen in [Figure 4.2](#) of the price changes for AAPL, and the 1/3 and 2/3 quantiles of this distribution were used as a cut-off point to determine how actions should be classified.

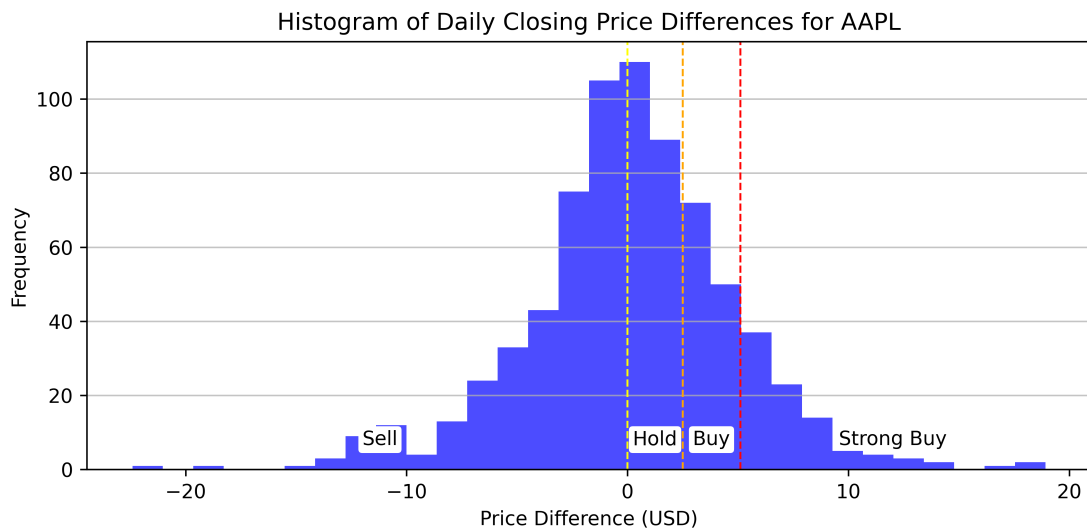


FIGURE 4.2: Histogram of daily closing price differences for AAPL.

Using these thresholds, each price change was categorised into one of four action categories, as seen in [Table 4.1](#).

TABLE 4.1: Action Categories Based on Price Change

Action	Name	Description
-1	Sell	A negative price change, indicating that the algorithm should sell or avoid taking a position in the stock.
0	Hold	A small positive price change, suggesting a hold decision, where no immediate action is taken.
1	Buy	A moderate positive price change, implying that it is reasonable for the algorithm to initiate a buy position.
2	Strong Buy	A significant positive price change, representing a strong buy signal where the model should confidently take a position.

4.2.2 Model Development

The training data defined in Section 3.4.3 was further split into a 60–40 train–test split. This split allowed the model to be tested on unseen data during development, which enables the model to learn and update its parameters during training. Standard techniques such as k-fold cross-validation [3] were avoided due to the temporal nature of financial data, where shuffling would introduce data leakage. Instead, a time-based split was used to preserve chronological order and maintain the integrity of the trading environment.

A moderately deep MLP network was selected, consisting of three hidden layers with decreasing node sizes. Given that the dataset included nineteen features on seven hundred and thirty-four sequential days, increasing the depth of the model beyond this point would have risked over-fitting, particularly due to the relatively limited number of training samples. The use of a tapered architecture, which is where the dimensionality is gradually reduced across layers, helps compress the learnt representation, and encourages the model to retain only the most salient patterns. This reduction also acts as a form of regularisation, forcing the network to prioritise meaningful signals over the memorisation of noise.

Several activation functions were tested during development. Although the *tanh* function initially performed well in setups without the *L-BFGS* solver [84], the resulting models failed to learn useful decision boundaries in this context. This may be due to *tanh*'s saturation effects and weaker gradient propagation in deeper networks, especially with small datasets [22]. In contrast, when the solver was switched to *L-BFGS* which is a second-order optimiser well suited for smaller datasets, the *ReLU* activation yielded significantly better results. *ReLU*'s non-saturating behaviour allowed the model to converge better, and *L-BFGS* provided stable updates without the noise of stochastic optimisers.

The final configuration of the model, summarised in Table 4.2, was applied consistently across all stocks. Although fine-tuning each model individually was possible, this would not have been aligned with the RL approach, where a single policy network is trained across multiple assets. In addition, the chosen configuration demonstrated consistently strong performance, making stock-specific tuning unnecessary. This approach also ensured that any performance variation could be attributed to the behaviour of the asset rather than arbitrary hyperparameter differences.

TABLE 4.2: MLP Hyperparameters Used for Supervised Learning

Parameter	Value
Hidden Layers	(32, 16, 8)
Max Iterations	5000
Alpha (L2 penalty)	1×10^{-4}
Learning Rate	1×10^{-4}
Activation Function	ReLU
Solver	L-BFGS

L2 regularisation, also known as weight decay, was applied consistently in all models, using a penalty term $\alpha = 1 \times 10^{-4}$. This technique discourages the model from assigning large weights to any particular input feature, helping to prevent over-fitting. By penalising large weights, L2 regularisation smooths the learnt function, thus improving the stability of training. In earlier experiments, omitting this regularisation showed highly variable behaviour and increased variance between training and test performance. In contrast, overly strong regularisation, example $\alpha \geq 10^{-2}$, caused the network to under-fit, producing overly conservative predictions that failed to respond appropriately to market shifts. [85]

4.2.2.1 Temporal Window Configuration

As discussed in Section 3.4.3, the input to the model was structured using a rolling window of historical data. Several window sizes were tested during development, including lengths of 3, 5, 10 and 15 days. Longer windows were avoided, as they would have overly smoothed the input features and risked diluting relevant short-term patterns. To compare different configurations, a series of experiments were conducted in which the model was trained and evaluated under each window length, respectively. The results show how the performance of the model varies with different window sizes as seen in Figure 4.3.

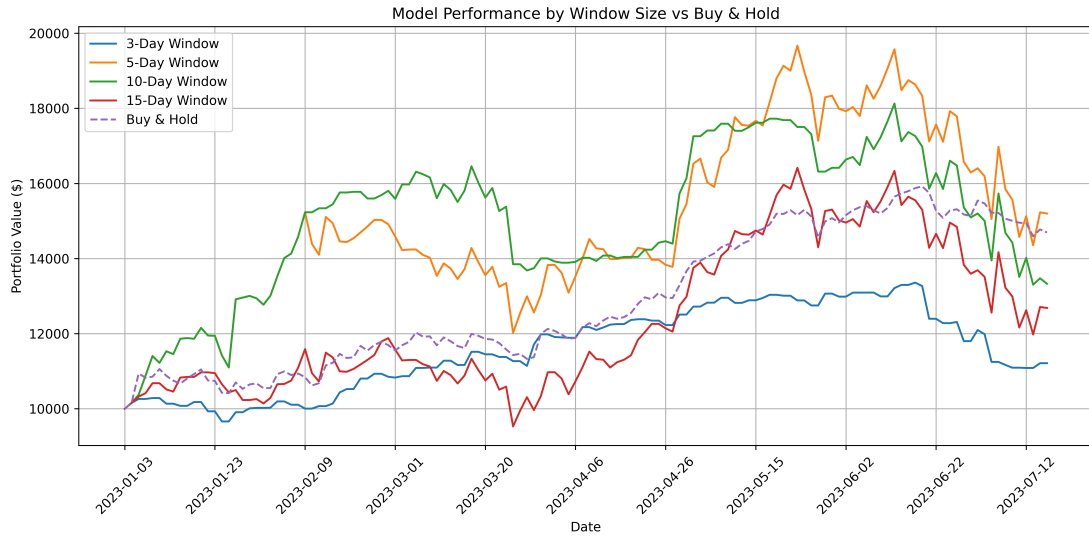


FIGURE 4.3: Model performance with different window sizes compared to the buy-and-hold strategy

The 5-day window consistently gave the best results in terms of both validation accuracy and trading simulation, so it was chosen to be used throughout the rest of the project. A 3-day window did not capture enough information, while 10- and 15-day windows smoothed out the volatility too much and made the signals less responsive.

4.2.3 Evaluation Metrics and Performance Analysis

Once training was complete, each model was evaluated using a confusion matrix as seen in Figure 4.4, which visually shows the number of correct and incorrect predictions for each action class.

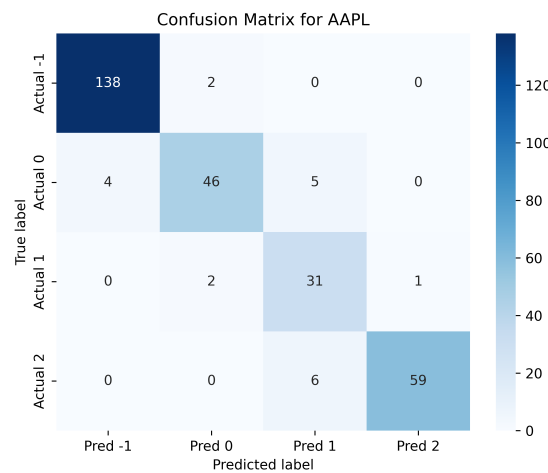


FIGURE 4.4: Confusion matrix for AAPL, showing the number of correct and incorrect predictions for each action class.

For AAPL, the model demonstrated strong performance in predicting class -1, with 138 correct classifications and minimal confusion with other labels. The class 0 and the class 2 also showed reasonable precision, although some misclassification into adjacent categories suggests a mild uncertainty around moderate movements.

How well the model was trained could not be assessed purely through training and testing accuracy, since high accuracy did not necessarily reflect correct label classification. What mattered most was the nature of the errors. If the model misclassified a -1 as a 1 or 2, it would take the opposite trading action, which could result in serious losses. On the other hand, confusing a 1 with a 2 wasn't as damaging, since both suggested buying the stock.

Optimising the network results across all stocks proved challenging. Parameter settings that minimised misclassification in one asset often performed poorly in others due to varying volatility profiles and the randomness in model initialisation. Even with identical hyperparameters, repeated runs could produce different behaviours. This variability highlighted the difficulty of training a supervised model across multiple financial assets.

4.2.4 Simulated Market Evaluation

The performance of the trained [MLP](#) models was evaluated in a simulated trading environment using historical price data from the validation period to replicate real-world investment conditions. The simulation began with a capital investment of \$10,000, equally allocated across all stocks. On the first trading day, when the portfolio did not hold positions, the available cash was used to purchase shares based on their current prices, ensuring full investment from the outset. Each subsequent trading day, the environment retrieved the current market conditions and provided the relevant features to the trained [MLP](#) models. Based on these inputs, each model predicted an action for its respective stock.

At each time step, the agent enters a rebalancing loop. The first part of this loop involves selling assets for which the model returns a negative signal. If a stock is predicted to decrease in value, all existing shares of that stock are liquidated, and the corresponding amount is added back to the cash balance. This ensures that the agent minimises losses or avoids holding positions in underperforming assets.

The agent identifies stocks with positive signals, either "Buy" or "Strong Buy". Allocation ratios are calculated by normalising the weights across all positively flagged stocks to determine how the current cash reserve should be distributed. Stocks with

stronger signals receive more capital. The agent invests the allocated amount into each stock by purchasing as many whole shares as possible based on current market prices.

The performance of the portfolio was tracked daily, recording both cash holdings and the value of each stock position. At the end of each simulated trading month (assuming 21 trading days), the monthly return was calculated and recorded. This provided a view of how the model predictions translated into real market outcomes over time.

At the end of the simulation, the cumulative portfolio value produced by the [MLP](#) predictions was plotted alongside the buy-and-hold baseline as seen in Figure 4.5.

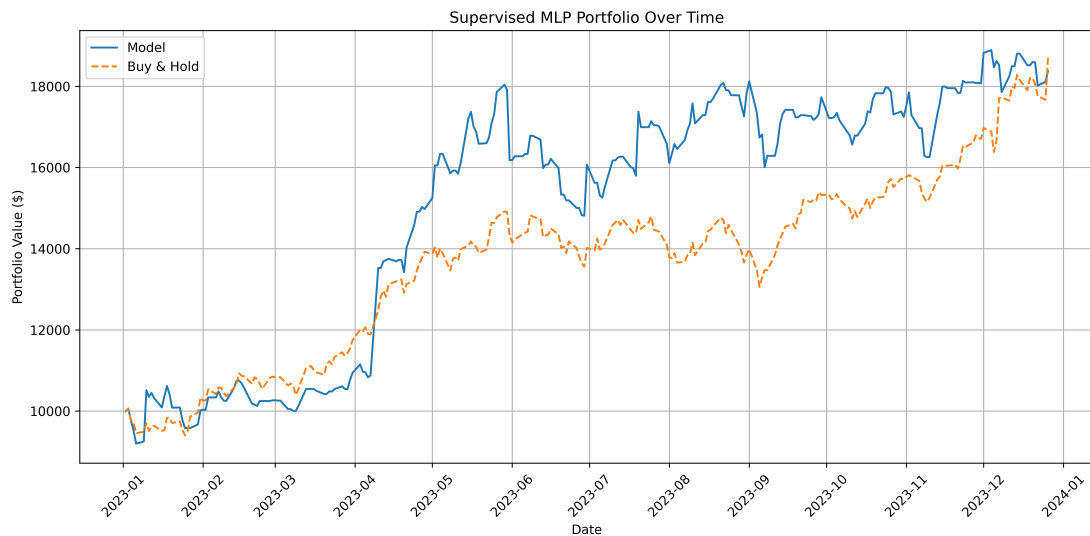


FIGURE 4.5: Portfolio value over time for the supervised MLP model compared to the buy-and-hold strategy.

In the simulation results shown in Figure 4.5, the supervised [MLP](#) agent initially performs on par with the buy-and-hold baseline. Despite a slight sideways movement between February and March 2023, the model avoids significant drawdown, suggesting that its early allocations were relatively stable. A sharp increase in portfolio value occurs in April, clearly outperforming the buy-and-hold benchmark. This surge is consistent with [MLP](#) issuing a "Strong Buy" signal for one or more of the best performing assets, such as META or TSLA, both of which show a rapid price increase during this period in Figure 4.1. This timely allocation demonstrates the model's ability to correctly detect strong momentum patterns when they emerge clearly in the input features.

From April through June, the portfolio continues to increase, reflecting the bullish market. However, in the second half of the year, particularly in Q4, the portfolio enters a more stagnant phase. Although overall performance remains positive, occasional dips suggest that the model may have failed to issue timely "Sell" signals on declining assets

or misclassified the momentum of other stocks, which is a limitation when using static thresholds for changing market behaviour.

The final portfolio value at the end of 2023 remains above the buy-and-hold benchmark, indicating that even though **MLP** does not continuously outperform it, it was able to capitalise on a well-timed high confidence signal that carried lasting benefits. In contrast, had the agent allocated a large part of the portfolio to a declining stock, the model would have incurred a loss and would have been hard to recover. These actions highlight the model's ability to capitalise on clear, large-scale trends, while also revealing its limitations in adapting to more volatile or unpredictable market conditions.

4.3 Reinforcement Learning Implementation

Building on the supervised learning foundation, this phase of the project focused on designing a **RL** agent capable of developing trading strategies through direct feedback with a simulated market environment. Although the supervised **MLP** model offered a static prediction-based framework, **RL** enables the agent to make sequential decisions in response to market feedback, optimising its trading policy over time.

To meet the unique challenges in financial markets, an actor–critic architecture was selected. As discussed in Section 2.5.5, this structure combines the strengths of value-based and policy-based approaches by maintaining two separate networks. In such volatile markets, immediate decisions must be made while also considering future outcomes; hence, by combining policy and value networks, the agent can learn better strategies that balance short-term actions with long-term portfolio gain.

An on-policy learning framework was adopted to maintain consistency between the evolving policy and the training trajectories used. As outlined in Section 2.5.2, off-policy methods often offer higher sample efficiency, but can struggle in unpredictable environments where past experiences quickly become outdated. In contrast, on-policy methods ensure that updates are based on the latest market data, making them more suitable for fast-changing financial applications.

Among the family of on-policy actor–critic algorithms, **PPO** was selected because, as detailed in Section 2.5.5.3, **PPO** modifies standard policy gradient methods by introducing a clipped surrogate objective. This clipping prevents excessively large updates to the policy, which makes **PPO** particularly well suited to environments that are volatile. Compared to other actor–critic algorithms such as vanilla policy gradients or A2C, **PPO** maintains a favourable balance between exploration and convergence speed.

Financial markets differ significantly from classical [RL](#) domains like games or robotics because they are inherently non-stationary and noisy. Moreover, unlike infinite-horizon scenarios assumed by traditional [RL](#) algorithms, portfolio management tasks usually have finite investment horizons and require strategies that adapt to changing market conditions and perform well on unseen data, rather than simply capturing patterns in the training period. [PPO](#) incremental update mechanism is particularly advantageous here, as it reduces the risk of over-fitting by ensuring gradual and controlled improvements in the policy.

Furthermore, [PPO](#) easily accommodates different neural network architectures. This flexibility was crucial in systematically evaluating which architecture best captures temporal dependencies and market dynamics. In addition, [PPO](#)'s modular implementation aligns seamlessly with the structured, episodic training approach adopted in this project, enabling continual adaptation as new market data become available. This adaptability is essential in financial applications, where there is ongoing retraining to help the agent maintain performance and responsiveness to changing market conditions.

4.3.1 Data Preprocessing

As described in Section 3.4.3, the [RL](#) models were trained on sequences of historical market data structured as rolling windows. Based on the analysis in Section 4.2.2.1, a 5-day window was selected for this implementation.

Although raw price values were used in the supervised models, [RL](#) implementations require more feature processing due to the sensitivity of neural network gradients to the distribution and scale of input values. In supervised learning, raw values can still be useful since the model learns direct associations between input patterns and predefined labels, provided that the label generation process aligns with the feature scale. However, [RL](#) agents operate in sequential decision-making order, where the scale of features affects the evolution of the policy over time. Significant differences in asset prices can distort the learning process by giving undue weight to higher priced inputs. This issue is evident in Figure 4.1, where NVDA's price movements are barely shown when plotted on the same scale as MSFT.

To mitigate this issue, all inputs were standardised using z-score normalisation to ensure consistent gradient updates and balanced learning across all stocks throughout training. This preserves the relative variation within each asset while avoiding cross-stock bias. Other normalisation methods discussed in Section 3.4.2, such as Min–Max scaling, were not used because, as shown in Figure 3.4, they compress the data into a narrow range $[0, 1]$, flattening important dynamic movements. Similarly, normalising by the

final closing price causes all trajectories to converge around 1.0, eliminating much of the volatility essential for agent learning.

Figure 4.6 shows how z-score normalisation brings all stock prices to a comparable scale while preserving key patterns such as trends, spikes and corrections, which is essential to learn fine-grained trading strategies.

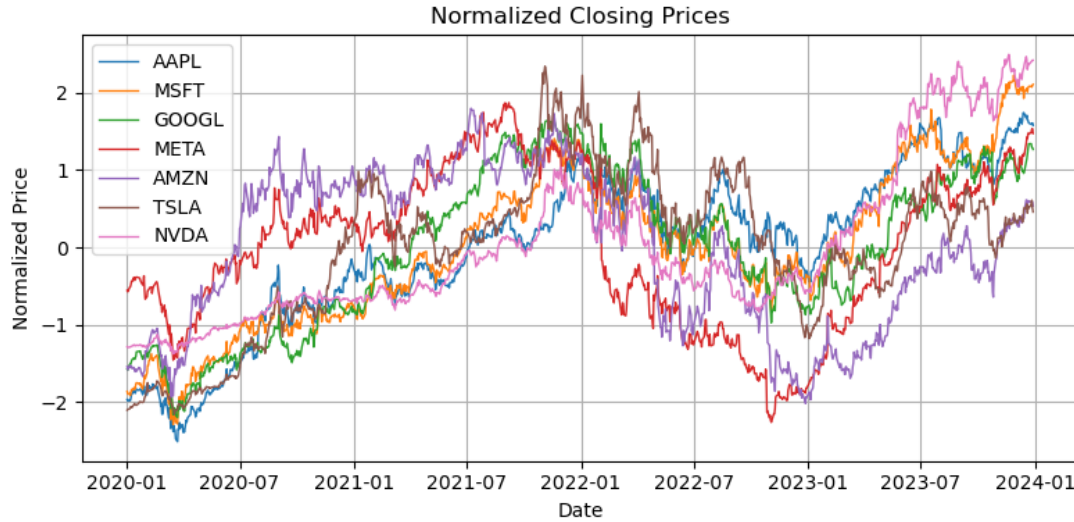


FIGURE 4.6: Z-score Normalised Closing Prices for Selected Stocks

4.3.1.1 Experimental Data Structure

To assess whether a single input or multiple features are sufficient for the models to learn and stabilise their policies, two input configurations were designed for each network. The first used only the normalised closing price, while the second input normalised the dataset that included the closing price and the market features introduced in Section 3.4.3. In supervised learning, feature engineering helps the model find useful patterns in the data. But in [DRL](#), the agent aims to learn directly from interacting with the environment, raising the question of whether adding additional features has the potential to improve results. However, the impact of additional inputs depends on the model architecture and the relevance of the signals provided. Excessive or redundant features may disrupt performance by reducing convergence quality. This experiment evaluates whether additional inputs enhance learning or interfere with policy stability.

4.3.2 System Overview

Before introducing the technical specifications of the [PPO](#) algorithm and the agent's training loop, it is essential to understand the overall structure of the [RL](#) system.

Figure 4.7 builds on the MDP foundation introduced in Section 2.4.1 and illustrates the full architecture adopted in this work.

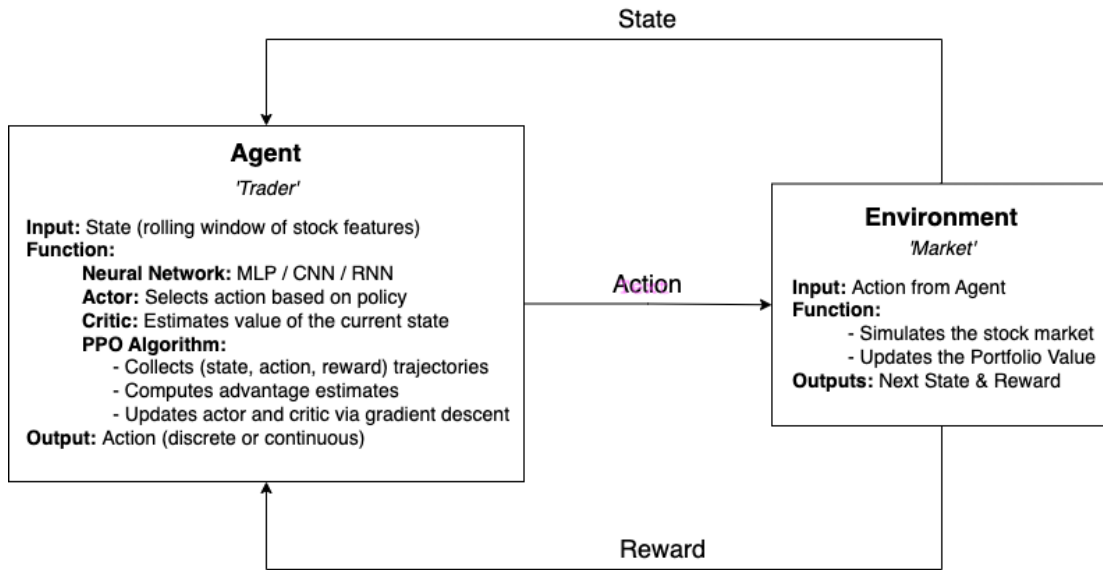


FIGURE 4.7: Reinforcement Learning Architecture: Agent-Environment Interaction

The system operates as a closed feedback loop between the agent and the environment. At each time step, the environment provides the agent with a state represented as a rolling window of the recent market features discussed in Section 4.3.1.1. The agent, through its policy network, processes the state and outputs an action which can be either a discrete decision, to buy, sell, or hold, or a continuous action indicating the percentage of the portfolio to allocate to an asset. This action is executed within the environment, leading to a change in portfolio holdings and an updated cash balance. The environment returns the next state and a scalar reward signal, representing the immediate financial outcome of the action taken.

The agent follows the actor-critic structure described previously in Section 2.5.5. The actor network determines the trading policy $\pi_{\theta}(a_t | s_t)$, dictating which actions (trades) to execute based on current market conditions. The critic estimates the value estimate $V_{\phi}(s_t)$, which aims to predict future returns from a given state. The rewards from the environment guide the iterative refinement of both networks. The critic's estimates help the actor network stabilise its policy updates by calculating the advantage, which measures how much better or worse an action was compared to the critic's expectation.

This setup frames trading as a sequential decision-making problem, where the agent continuously interacts with the environment by observing market states, executing trades, and receiving feedback through returns. Unlike the supervised learning approach in

Section 4.2, the actor–critic approach adapts dynamically to changing market conditions, allowing the agent to adjust strategies based on new data.

4.3.2.1 Reward Function Design

During the early design phase, various reward structures were explored, including cumulative multi-day returns, log returns, and Sharpe adjusted metrics. Although these offered the potential to capture broader performance insights, they also introduced additional noise and required smoothing techniques that complicated the learning process.

To maintain a clear and responsive learning signal, the design ultimately settled on a simpler approach: using the difference in the stock closing price from one day to the next as the reward. This method provides a direct link to short-term market movements, allowing the agent to adapt quickly during training without being influenced by portfolio-level fluctuations or long-term performance noise.

4.3.2.2 Stochastic Action Sampling for Portfolio Allocation

Unlike the discrete action space used in the supervised learning setup described in Section 4.2, the RL agent’s action operates within a continuous action space. Each action is a real-valued number representing the proportion of capital to allocate to a specific asset. Rather than making a single categorical decision, ‘Buy’ or ‘Strong Buy’ the agent directly decides how much to invest in each asset as a fraction of its total cash. Positive actions are normalised into ratios, which ensures that the portfolio stays fully invested without exceeding the available cash.

These continuous actions are sampled from a Gaussian distribution $\mathcal{N}(\mu_t, \sigma_t)$ where the mean μ_t and log standard deviation σ_t are obtained from the actor’s output. This allows the agent to produce a range of allocation values rather than being constrained to fixed discrete steps. As a result, the agent naturally introduces variability in its decisions, which is necessary to discover the best strategies.

As mentioned in Section 2.4.1.4, in RL it is crucial to maintain a balance between exploration and exploitation. In this PPO implementation, no explicit exploration strategies were applied as exploration arises naturally from stochastic policy. Even when the agent has high confidence in a given allocation, sampling from the Gaussian distribution ensures that nearby alternatives are still occasionally tested. This inherent variability encourages the agent to explore different strategies without requiring additional exploration mechanisms. This helps prevent premature convergence to suboptimal behaviours and supports continued learning throughout training.

By relying on a stochastic policy, the agent achieves an inherent balance between exploring new strategies and exploiting known profitable actions. This simplifies the training process while preserving the necessary adaptability to perform well in volatile financial markets.

4.3.3 Proximal Policy Optimisation Agent Architecture

This section outlines the design and development of the **PPO** agent, which is built on an actor–critic framework, as introduced in Section 4.3. The architecture is structured to support easy integration and experimentation with different types of neural networks, allowing modifications without requiring major changes to the underlying **PPO** algorithm. This modular design allows the easy substitution of different types of networks, including **MLPs**, **CNNs**, and **RNNs**, based on the specific input data used in each experiment. A systematic comparison of these architectures and their relative performance is provided in Section 4.4.

4.3.3.1 Actor–Critic Network Overview

The actor and critic networks in the **PPO** agent share a common structure up to their final layers. Both receive a state input s_t , followed by the hidden layers of the actor and critic networks, and are split into their final output layers, as seen in Table 4.3.

TABLE 4.3: Overview of the Actor and Critic Neural Network Architecture

Component	Description
Input	s_t which is a rolling window of input features
Hidden Layers	Configurable architecture supporting MLP, CNN, or RNN models depending on the experiment.
Actor Output	μ_t of the action distribution and a trainable $\log \sigma$.
Critic Output	$V_\phi(s_t)$ representing the expected return from s_t .

The actor and the critic are distinct components, each serving a specific function. The actor focusses on determining the optimal trading actions, while the critic estimates the expected return, guiding the actor’s policy updates. Both networks are trained simultaneously, allowing efficient policy optimisation without compromising the learning of either component.

4.3.4 Proximal Policy Optimisation Training Framework

4.3.4.1 Advantage Estimation and Critic Loss Computation

The training process follows a structured temporal organisation in which each episode represents a complete chronological progression throughout the market data described in Section 3.4.3. In PPO, the dataset is divided into batches during an episode, which are smaller and manageable subsets of the dataset. This batch approach is fundamental to the PPO implementation, as it updates the network in batches of transitions rather than at each individual time step or at the end of the dataset.

The training follows an epoch structure, where each epoch constitutes a complete pass through all batches derived from the collected experience. Once training starts, the trajectories are collected by sampling actions from the current policy and observing the resulting rewards and next states. At each time step t , the agent receives a reward r_t , obtained from the reward function of the environment R , discussed in Section 2.4.1.2, and computes the *TD error*, denoted by δ_t , which quantifies the discrepancy between the predicted value and the observed return as given by:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (4.1)$$

Where, $V_\phi(s)$ is the critic network's approximation of the true value function $V^\pi(s)$ discussed in Section 2.4.1.2.

The TD errors are accumulated over each batch and used to compute the advantage estimates using the **Generalised Advantage Estimation (GAE)** approach. GAE was chosen over simpler estimators, such as the 1-step TD error or Monte Carlo returns, because it strikes a balance between bias and variance. In highly volatile financial markets, the use of raw TD errors would introduce excessive variance, leading to unstable learning, while relying on full Monte Carlo returns would result in a slow response to recent changes. GAE introduces a smoothing parameter λ , allowing the advantage estimate to interpolate between low-variance but biased estimators and unbiased high-variance ones. In this work, λ was set to 0.95, a value commonly used in RL applications to provide a good balance between stability and responsiveness. This choice helps the agent smooth out noisy financial rewards without overly dampening its ability to react to new market information.

The parameter $\gamma \in [0, 1]$ controls the agent's emphasis on future rewards versus immediate rewards. A higher γ (close to 1) encourages the agent to plan further into the future, while a lower γ prioritises short-term rewards. Following standard practices

in financial [RL](#), γ was set to 0.9, to ensure that the agent properly balances short-term profitability with long-term portfolio growth [86].

The critic network is trained to predict the expected return from a given state, as discussed in Section 4.3.2. To update this estimate, a target value is computed using information on the observed rewards. This target, denoted by V_t^{target} , serves as a more accurate return estimate and is used to minimise the value loss during critic training.

To compute V_t^{target} , the advantage is first estimated at each time step t . This advantage captures how much better (or worse) the actual return was compared to the critic's baseline prediction. The advantage estimate is computed according to the following:

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l} \quad (4.2)$$

This advantage is then combined with the critic's current value estimate to form the target:

$$V_t^{\text{target}} = \hat{A}_t + V_\phi(s_t) \quad (4.3)$$

This formulation encourages the critic to better approximate actual returns by leveraging both the observed reward signals and the temporal structure of the trajectory. It also enables the actor to refine its policy updates using advantage values that reflect meaningful deviations from expected performance.

When these computations are completed, all the trajectories across different stocks and days are flattened into a unified dataset. This transformation collapses both the time and the asset dimensions, allowing the use of standard batch [Stochastic Gradient Descent \(SGD\)](#) techniques during training.

Random batch sampling is applied to the flattened dataset, with networks trained over multiple epochs on the same batch of experiences. Random sampling exposes the agent to a more diverse set of transitions in each update step, helping prevent over-fitting, while multiple passes over the same data allows the networks to internalise important patterns.

Instead of performing a single update after collecting new trajectories, the agent is trained over multiple epochs using the same batch of data. This allows networks to better internalise the information contained within each trajectory. The critic network is subsequently updated by minimising a mean squared error loss between its predicted state values and the calculated target returns, as given by:

$$L_{\text{critic}} = \frac{1}{T} \sum_{t=1}^T \left(V_\phi(s_t) - V_t^{\text{target}} \right)^2 \quad (4.4)$$

where, L_{critic} denotes the critic loss function, which quantifies the average squared difference between the estimated values and the target values for all the time steps of the batch. This regression objective encourages the critic to produce accurate, low-bias value estimates, which are crucial for reliable advantage estimation in subsequent actor updates. Reducing this error indirectly stabilises policy learning, as inaccurate value functions would otherwise inject noise into the policy gradients and degrade overall learning performance.

4.3.4.2 Policy Update

The actor network is updated using the clipped surrogate objective shown in Equation 2.19. Following the recommendations of Schulman et al. [38], the clipping threshold ϵ was set at 0.2. Smaller values of ϵ would overly restrict updates, significantly slowing learning, while larger values could destabilise training by allowing excessively large changes in the policy. By setting $\epsilon = 0.2$, the agent can make consistent and controlled improvements to its trading strategy.

Rather than directly maximising the standard policy objective, PPO constrains the magnitude of each update to ensure stable learning. The probability ratio between the current and old policies is calculated as follows:

$$P_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad (4.5)$$

This ratio measures how much the current policy π_{θ} deviates from the previous policy $\pi_{\theta_{\text{old}}}$ when selecting the same action for the same state. A value of $P_t = 1$ indicates that the policies are identical for that action, while deviations from one signal how significantly the new policy has changed.

4.3.4.3 Training Algorithm Summary

The complete PPO training procedure is summarised in Algorithm 1.

Algorithm 1 Training Procedure for PPO Agent

```

1: Input: Initialised actor parameters  $\theta$ , critic parameters  $\phi$ 
2: for each stock  $s \in \mathcal{S}$  do
3:   for each batch epoch  $e = 1$  to  $N$  do
4:     for each time step  $t = 1$  to  $T$  do
5:       Predict action distribution parameters,  $\mu_t$  and  $\sigma_t$ , from actor
6:       Sample action from distribution
7:       Execute action in environment
8:       Observe next state and reward
9:       Estimate values from critic
10:      Compute TD error using Equation 4.1
11:    end for
12:    Calculate GAE using Equation 4.2
13:    Calculate Value target using Equation 4.3
14:    Minimise critic MSE loss using Equation 4.4
15:    Compute probability ratio between current and old policy using Equation 4.5
16:    Save current actor policy as old policy
17:    Calculate clipped surrogate objective using Equation 2.19
18:    Perform gradient ascent to update actor and critic
19:  end for
20: end for

```

4.4 Neural Network Architecture Comparison

The following section builds on the actor–critic framework described in Section 4.3.3.1 by integrating and comparing three distinct network architectures within the same PPO agent to assess their respective strengths in portfolio allocation. Each network offers different advantages, allowing a good comparison of their suitability for the data and the implementation of the PPO framework used in this project. Specifically, MLPs serve as a fully connected baseline for high-dimensional function approximation [28], CNNs leverage spatial correlations across rolling windows of market features [37], and RNNs capture the temporal dependencies inherent in sequential price data [33]. All architectures

are trained and evaluated using identical data splits and window configurations as seen in Section 4.3.1, to ensure a controlled comparison within a single PPO framework.

4.4.1 Multilayer Perceptron Architecture

Figure 4.8 shows the MLP architecture used in this study. An input s_t is first passed through two fully connected layers, each followed by a ReLU activation, thereby enabling the network to learn non-linear feature combinations. The output of the second layer feeds into a third fully connected layer, whose outputs diverge into two heads: the actor's μ_t and $\log \sigma_t$. The critic network shares the same first three layers but uses a separate final head to output $V_\phi(s_t)$.

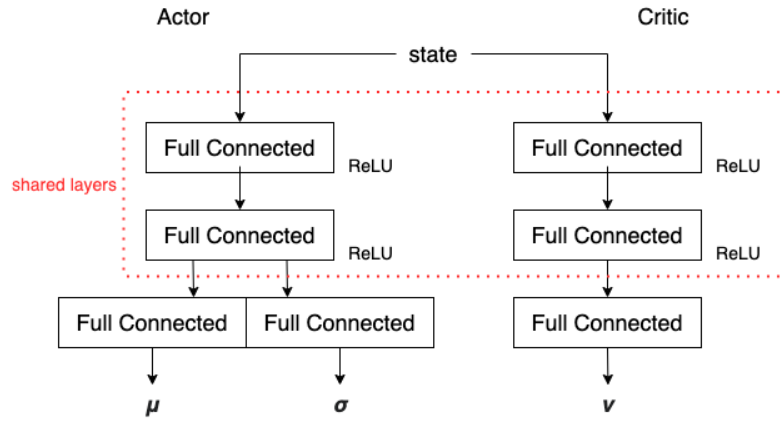


FIGURE 4.8: Architecture of the MLP Actor–Critic Network

The three hidden layers offer sufficient capacity to capture the intricate relationships across the rolling windows of prices and indicators, without introducing optimisation challenges typically associated with deeper networks. ReLU activations are used after each layer to address the vanishing-gradient problems and improve convergence speed, as shown in [34]. The use of shared layers followed by separate actor and critic heads supports efficient feature reuse while preserving distinct optimisation objectives for policy and value estimation, a configuration shown to improve sample efficiency in on-policy RL methods [36].

4.4.1.1 With Closing Price as the State

For experiments using only the closing price as input, the MLP architecture was tuned with a separate set of hyperparameters to account for the reduced feature space. These values are summarised in Table 4.4.

TABLE 4.4: MLP using Closing Price PPO Agent Hyperparameters

Hyperparameter	Value
Hidden Layer Size	(16,16,16)
Optimizer	Adam
Learning Rate	3×10^{-4}
L2 Regularisation	1×10^{-4}
Epochs	200
Batch Size	32

A smaller hidden layer configuration was selected to prevent over-fitting, given the limited input dimensionality. Adam optimiser was used with a learning rate of 3×10^{-4} , which is a common choice in RL due to its ability to adaptively scale learning rates and handle noisy gradient updates. The L2 regularisation was applied as mentioned in Section 4.2.2.

The training curves for this configuration are shown in Figure 4.9. The plot on the left displays the actor loss, which corresponds to the clipped surrogate objective in PPO as defined in Equation 2.19. This loss reflects the negative expected advantage weighed by the probability ratio between the new and old policy distributions [38]. The curve shows a smooth and stable downward trend, converging around epoch 50 and plateauing near -0.075 , indicating consistent and controlled policy updates.

The plot on the right of Figure 4.9 plots the average log of P_t defined in Equation 4.5 as:

$$\frac{1}{B} \sum_{i=1}^B \log P_t,$$

which measures the deviation between the updated policy and the previous policy in a batch of sampled actions. In the context of PPO, the logarithmic ratio serves as a direct understanding of the policy stability. Ideally, the average $\log P_t$ should remain close to zero to indicate that the updates are staying within the trust region imposed by the clipping mechanism. Minor deviations from zero in the log-ratio are expected, as they reflect incremental and meaningful updates to the policy.

In this implementation, the average log-probability initially declines as the agent explores the action space more randomly, which is expected during the early stages of training when the policy is still unrefined. As learning progresses and the agent begins to favour more consistent, higher-reward actions, the log-ratio gradually increases and stabilises. The eventual plateau at a slightly negative value indicates that the policy

continues to adapt, but in a stable and constrained manner. This behaviour aligns with the goal of the PPO framework ensuring steady improvement while preventing large, destabilising policy updates.

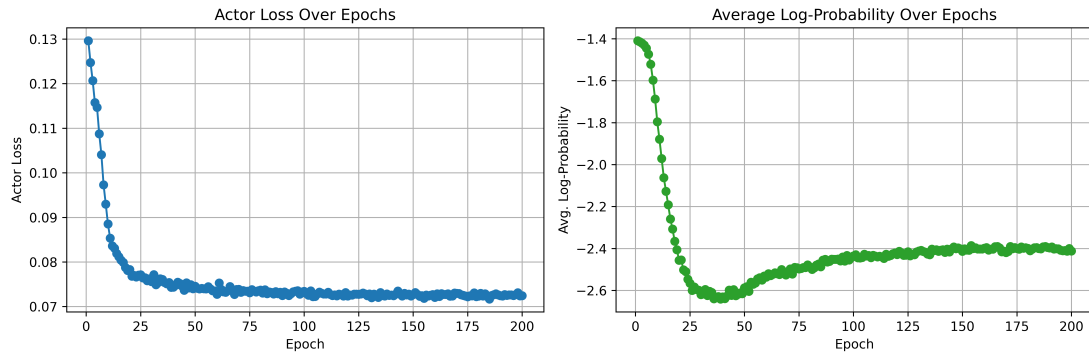


FIGURE 4.9: Actor loss (left) and average log-probability (right) for MLP architecture trained on closing price input.

4.4.1.2 With Features as the State

When using the normalised market indicators with the normalised closing prices, the MLP was re-tuned to better accommodate the increased input dimensionality and richer feature space. Table 4.5 lists the selected hyperparameters.

TABLE 4.5: MLP using Features PPO Agent Hyperparameters

Hyperparameter	Value
Hidden Layer Size	(32,32,32)
Optimizer	Adam
Learning Rate	1×10^{-4}
L2 Regularisation	1×10^{-4}
Epochs	500
Batch Size	32

Compared to the previous configuration, the learning rate was reduced to 1×10^{-4} and the hidden layer size was increased to 32 to allow more stable updates given the increased variance introduced by multiple input features.

In Figure 4.10, the actor loss decreases rapidly during the initial epochs and gradually plateaus around 0.085. Simultaneously, the average log-probability drops sharply in the initial stages, reflecting uncertainty during exploration and levels off as the model

becomes more confident in its decisions. This pattern suggests successful learning and increases consistency in the agent’s behaviour. However, the average log-probability begins to decline again after epoch four hundred, suggesting that the agent’s confidence in its actions is decreasing. This is likely due to over-fitting caused by excessive training epochs on the same set of trajectories, a common issue in [PPO](#) when the policy is updated too many times per batch.

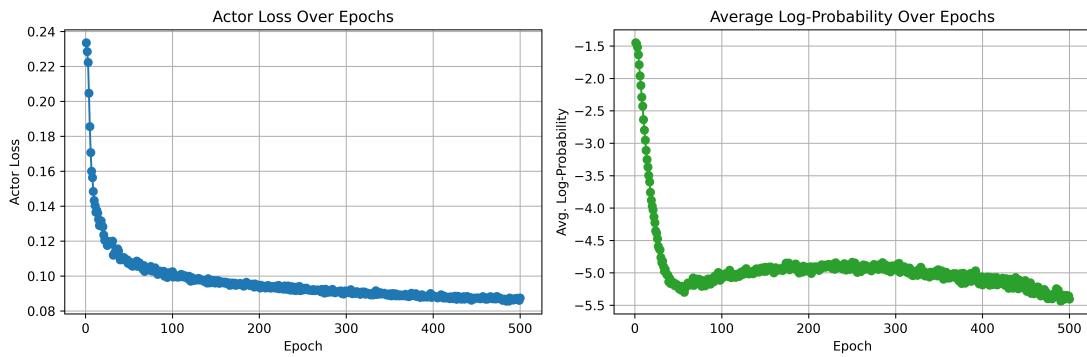


FIGURE 4.10: Actor loss (left) and average log-probability (right) for the MLP architecture with market features as input.

4.4.2 Convolutional Neural Network Architecture

Figure 4.11 illustrates the [CNN](#) architecture implemented in this project. Each state input is processed through a stack of three 2D convolutional layers, each followed by instance normalisation and a [ReLU](#) activation function. This design allows the network to extract local temporal and cross-asset patterns from the rolling input window.

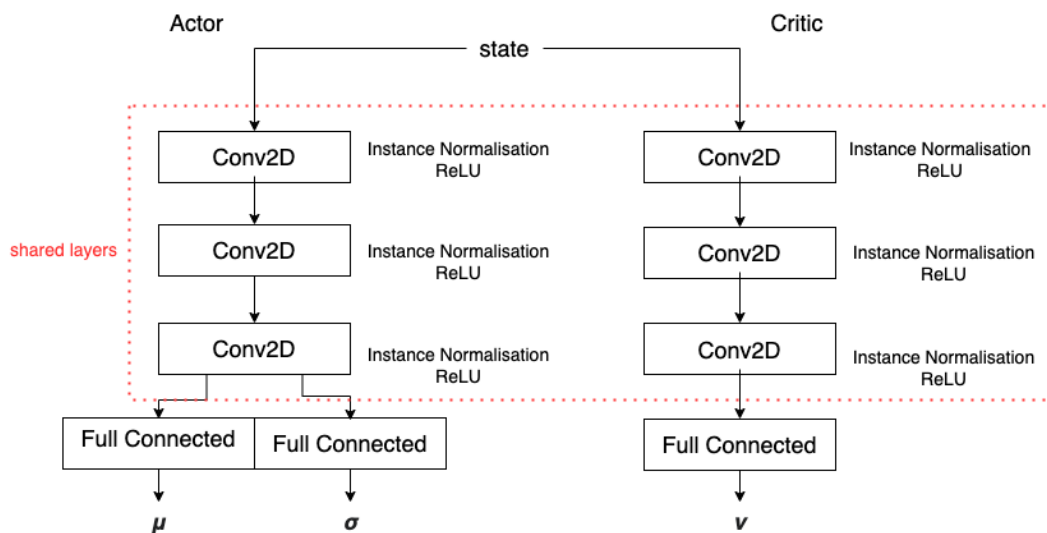


FIGURE 4.11: Architecture of the CNN Actor-Critic Network

The use of convolutional layers in place of fully connected ones introduces spatial inductive bias, making the network more sensitive to local feature interactions while reducing the total number of trainable parameters. Instance normalisation is applied following each convolution instead of batch normalisation [87]. This decision comes from the consistently small batch sizes and fixed window lengths used during training. Batch normalisation depends on accurate statistics computed across an entire batch, which can become unreliable when batches are small. In contrast, instance normalisation normalises each sample independently, providing more stable training dynamics under these constraints. This helps reduce internal covariate shift and maintain consistent feature scaling throughout training. The final convolutional output is passed to the actor and critic heads to produce the policy and value estimates.

4.4.2.1 With Closing Price as the State

Table 4.6 lists the hyperparameters used to train the CNN agent on closing price data. The *AdamW* optimiser was selected for its improved handling of weight decay compared to standard Adam, allowing for a more principled decoupling of regularisation from gradient updates [88]. A slightly higher learning rate was used to encourage faster convergence, while a batch size of 64 was selected to improve gradient estimation stability given the convolutional structure. A total of 700 epochs were used to give the CNN agent enough time to stabilise its learning process. Compared to MLPs, CNNs exhibited slower and more variable convergence behaviour, probably due to their architectural focus on local features. Extending the duration of the training helped mitigate this and allowed the agent to gradually refine its policy over time.

TABLE 4.6: CNN using Closing Price PPO Agent Hyperparameters

Hyperparameter	Value
Hidden Layer Size	(32, 32, 32)
Optimizer	AdamW
Learning Rate	5×10^{-4}
L2 Regularisation	1×10^{-4}
Epochs	700
Batch Size	64

The training results for the CNN-based PPO agent with closing price as state are shown in Figure 4.12. The actor loss exhibits an overall downward trajectory, but with

persistent variability and frequent spikes, particularly between epochs 100 and 600. Despite these fluctuations, the loss remains relatively contained between 0.05 and 0.06, indicating that the updates, while noisy, do not completely destabilise learning. This suggests that while learning progresses, policy updates remain unstable and sensitive to short-term fluctuations in the input data. The instability likely stems from the [CNN](#)'s architectural bias toward spatial locality, which limits its ability to capture long-range temporal dependencies. Although convolutional filters extract short-term features, they lack the recurrent structure needed to model critical sequential patterns in financial time series. Consequently, the agent may not form coherent temporal representations, resulting in noisier gradients and slower convergence.

The average log-probability curve initially spikes, followed by a gradual decrease during the early exploration phase. It gradually levels off near epoch 400 and begins to stabilise around -2.0. This behaviour is consistent with controlled and incremental policy updates. The final value is very close to that of the [MLP](#) agent trained in closing prices, as seen in Figure 4.9, which settles around -2.4, suggesting a comparable calibration between the two models. The relatively flat curve toward the end of training reflects that the policy no longer makes large shifts, implying that the [PPO](#) clipping mechanism successfully constrains updates as intended.

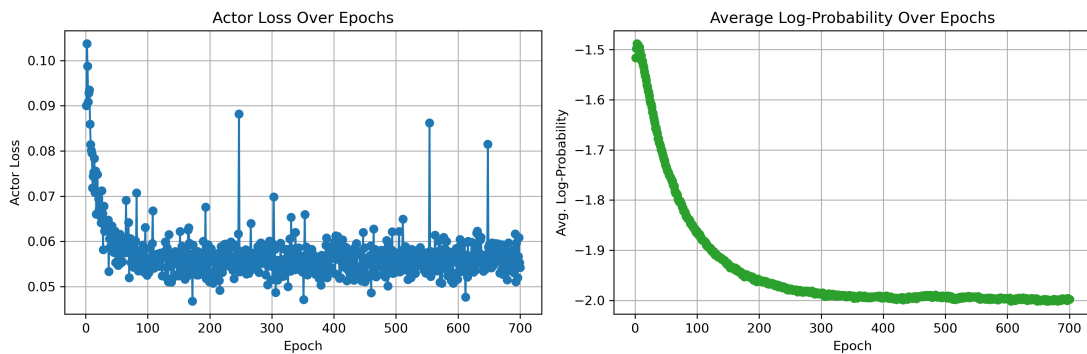


FIGURE 4.12: Actor loss (left) and average log-probability (right) for CNN architecture trained on closing price input.

4.4.2.2 With Features as the State

Table 4.7 presents the hyperparameters used to train the [CNN](#) agent on the full feature set. Given the expanded input space, the convolutional layers were increased to 128, 64, and 32 filters. This configuration allows the model to hierarchically extract and compress spatial patterns from input windows, enabling richer representation learning.

TABLE 4.7: CNN using Features PPO Agent Hyperparameters

Hyperparameter	Value
Hidden Layer Size	(128, 64, 32)
Optimizer	AdamW
Learning Rate	3×10^{-4}
L2 Regularisation	1×10^{-4}
Epochs	500
Batch Size	64

The actor loss decreases steadily over training epochs with relatively low variance, suggesting that the policy is improving consistently. Compared to models with fewer input features, this smoother descent implies that the additional features are contributing to more stable and informative policy updates. However, the loss continues to decline even at epoch 500, which indicates that the policy has not yet fully stabilised, which is possibly due to the increased complexity of the input space requiring more time to converge.

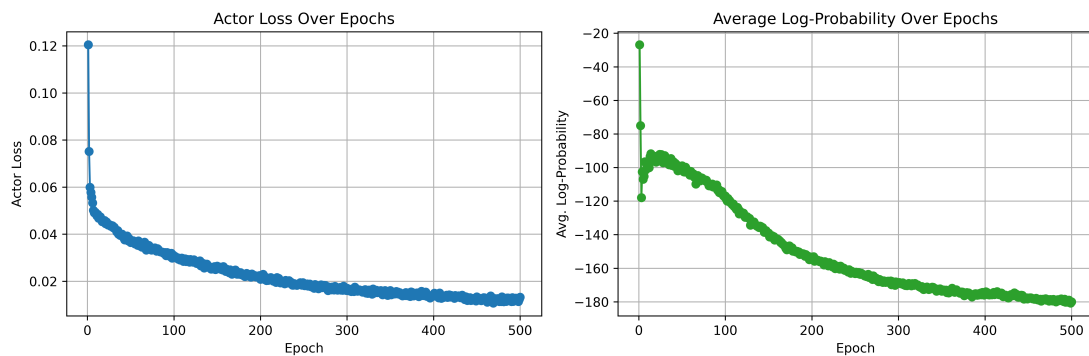


FIGURE 4.13: Actor loss (left) and average log-probability (right) for CNN architecture trained with market features input.

However, the average log-probability shows a sustained and pronounced decline to very low values, reaching below -170. This behaviour is atypical and suggests a growing mismatch between the sampled actions and the policy distribution, which is a sign of poor calibration. Rather than indicating improved confidence, the plot reflects degraded policy quality, possibly due to over-fitting or unstable gradients. This may stem from ineffective feature extraction or excessive reliance on patterns in training data that do not carry over well. In this case, the use of six market features as input may have added

unnecessary noise, making it harder for the CNN to pick out useful signals, hence it contributes to the instability seen during training.

4.4.3 Recurrent Neural Network

Figure 4.14 illustrates the vanilla RNN architecture implemented in this project. Each input state is first processed through a fully connected layer with a ReLU activation, projecting the input into a higher-dimensional feature space suitable for temporal modelling. This is followed by two stacked recurrent layers with $\tanh$ activations, allowing the network to capture short-term dependencies between sequential observations. Layer normalisation is applied after the recurrent layers to stabilise training by normalising hidden state activations across time steps, which helps mitigate gradient instability [89].

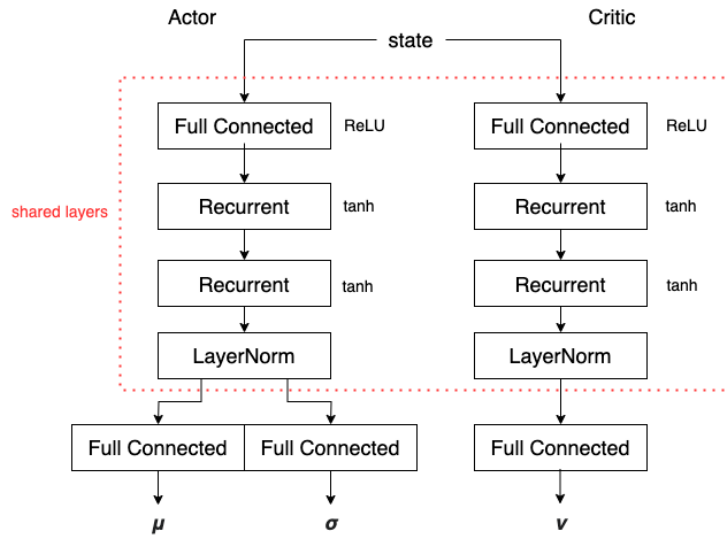


FIGURE 4.14: Architecture of the RNN Actor-Critic Network

4.4.3.1 With Closing Price as the State

Table 4.8 outlines the hyperparameters used to train the vanilla RNN agent on the closing price data. The network begins with a fully connected layer of 32 hidden units, followed by two recurrent layers of 16 hidden units each was selected to reduce the risk of over-fitting on the limited input space while retaining sufficient capacity to model short-term temporal dependencies. SGD was used as the optimiser to allow for more gradual and steady updates during training [90]. In contrast to adaptive methods such as Adam, SGD applies a constant update step, which can lead to more predictable learning dynamics in simpler or lower-dimensional problems. A larger batch size of 128 was

chosen, since RNNs converge slowly and any smaller batch size would have extended training times by hours.

TABLE 4.8: RNN using Features PPO Agent Hyperparameters

Hyperparameter	Value
Hidden Layer Size	(32,16,16)
Optimizer	SGD
Learning Rate	3×10^{-4}
L2 Regularisation	1×10^{-4}
Epochs	500
Batch Size	128

In Figure 4.15, the actor loss shows a steep decline in the first few epochs, followed by a long, flat plateau. This initial drop suggests that RNN rapidly learns to exploit short-term sequential patterns in the normalised closing price. However, subsequent stagnation indicates that the policy struggles to improve further, which is likely due to limited information in the input and architectural limitations. Specifically, the model uses a vanilla RNN, which lacks the gating mechanisms needed to retain information over longer sequences. As a result, it may be poorly suited to the temporal credit assignment demands of PPO, where longer-term dependencies between actions and rewards are crucial.

In the right plot, the average log-probability decreases sharply early on, reflecting increased exploration and growing policy confidence. After epoch 300, this trend reverses, with a slight upward drift suggesting that the policy begins to soften, potentially redistributing probability mass more evenly across actions. This behaviour may indicate that the policy was initially too deterministic and is now adjusting, possibly as a result of instability in value function updates or limitations in RNN's ability to capture longer-term patterns in the data.

Overall, while recurrent architectures are theoretically well matched to financial time series due to their ability to capture temporal dependencies, the use of a vanilla RNN in this PPO implementation appears insufficient. As mentioned in Section 2.3.3, standard RNNs are prone to exploding and vanishing gradient problems, significantly affecting their ability to retain and propagate long-term information. In practice, the literature rarely employs vanilla RNNs for these reasons. Instead, gated architectures such as Gated Recurrent Units (GRU) or LSTM, which are extensions of the RNN family, are

more commonly adopted. These models are specifically designed to address the gradient instability issues through gating mechanisms that regulate information flow, allowing them to maintain relevant signals over longer sequences.

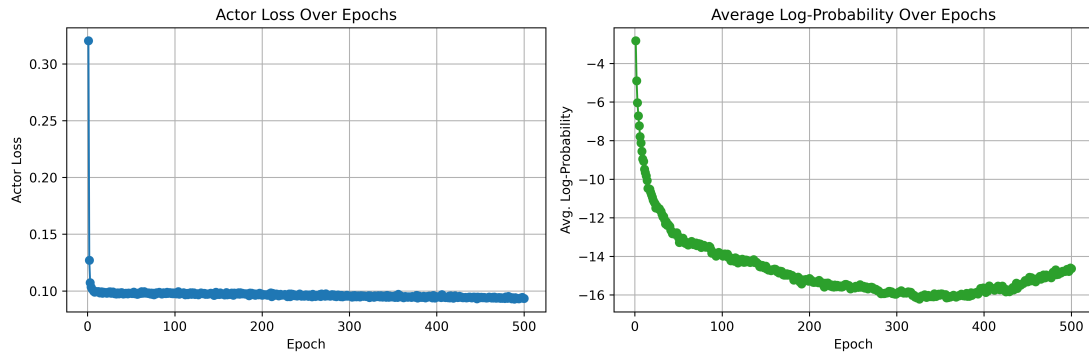


FIGURE 4.15: Actor loss (left) and average log-probability (right) for RNN architecture trained on closing price input.

4.4.3.2 With Features as the State

Table 4.9 presents the hyperparameters used to train the RNN-based agent on the full feature set. A deeper configuration of a fully connected layer of 64 hidden units, followed by two recurrent layers of 32 hidden units each to accommodate the increased input dimensionality. Despite the higher capacity, SGD was retained as the optimiser to maintain stable and gradual parameter updates.

TABLE 4.9: RNN using Features PPO Agent Hyperparameters

Hyperparameter	Value
Hidden Layer Size	(64, 32, 32)
Optimizer	SGD
Learning Rate	3×10^{-4}
L2 Regularisation	1×10^{-4}
Epochs	500
Batch Size	128

The results of the RNN model trained with additional market features further underscore the concerns discussed in Section 4.4.3.1. As shown in Figure 4.16, the actor loss decreases quickly and plateaus at a low level, while the average log-probability continues to fall steeply throughout training, reaching values below -2500. This trend

suggests that the policy becomes increasingly deterministic and poorly calibrated. The sharp decline probably results from the *RNN*'s inability to manage the added input features, especially in the absence of gating mechanisms. These findings support the argument that vanilla *RNNs* are not well suited to *PPO*'s temporal learning demands and should be replaced with architectures that provide better control over temporal information flow.

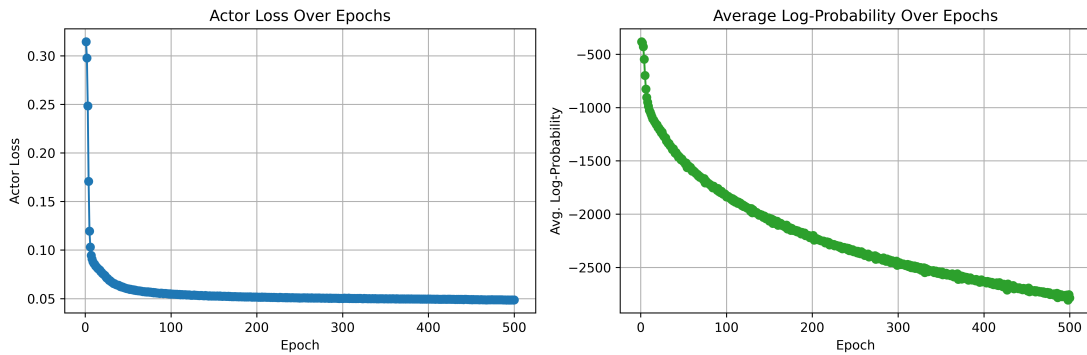


FIGURE 4.16: Actor loss (left) and average log-probability (right) for *RNN* architecture trained with market features input.

4.4.4 Summary of Model Tuning

Table 4.10 presents a comparative overview of the convergence behaviour and policy stabilisation in all configurations of the *PPO* model. The *MLP*-based models exhibit fast and stable convergence, while the *MLP Features* model exhibits the fastest policy stabilisation overall. *CNN* models also show relatively stable training dynamics, though they require more epochs to converge, especially when the state is the market features. Notably, the *MLP Closing* and *CNN Closing* configurations achieve similarly close final average log-probability values of approximately -2.4 and -2.0, respectively. This indicates that despite architectural differences, both models converge to policies with comparable confidence and entropy under low-dimensional input conditions. It also suggests that *CNNs* do not extract substantially more expressive policies than *MLPs* in this specific setting.

In contrast, *RNN*-based models, particularly with engineered features, show erratic or incomplete convergence. The extremely low log-probabilities observed in *RNN Features* and *CNN Features* may point to numerical instability, over-fitting, or excessive policy entropy in high-dimensional input spaces. The *RNN Features* model does not fully stabilise, reinforcing its limitations in handling the data structure used.

TABLE 4.10: Convergence and Policy Stabilisation Metrics for PPO Architectures

Model	Actor Loss Converged	Final Actor Loss	Policy Stabilised	Final Avg Log-Prob
MLP Closing	~75	0.072	~125	-2.4
MLP Features	~200	0.087	~100	-5.0
CNN Closing	~100	0.055	~300	-2.0
CNN Features	~300	0.015	~400	-180
RNN Closing	~25	0.098	~300	-15.5
RNN Features	~100	0.050	>500	-2700

4.4.5 Simulated Market Evaluation

Although Section 2.4 outlined the general role of environments in RL, this section details the custom market simulation environment developed for the PPO agent in this project. This implementation is an extension of the environment used for the supervised learning model in Section 4.2.4, with specific modifications to suit the requirements of the PPO agent.

Rather than using established libraries such as Gymnasium [91] or FinRL [70], a custom environment was created for this project. Although these libraries offer a variety of useful features, they often include additional elements such as transaction costs, slippage, short selling, and rebalancing rules. Although realistic, these features introduced unnecessary complexity for the way the PPO agent was designed in this work. In addition, many existing environments are built for discrete actions, but this project required continuous portfolio allocation. By developing the environment from scratch, it was possible to simplify the design and focus only on the relevant aspects needed to train the agent. This approach allowed greater control over the learning process and ensured that the environment was fully aligned with the assumptions made during training.

The environment was designed to mirror the basic portfolio management workflow. At each step, the agent observes a window of recent price movements, decides how to allocate capital across different assets, and receives feedback based on how the portfolio value changes. This feedback is used to update the agent's learning. The environment logic in any RL system is divided into two main functions: `reset()` and `step()`.

The `reset()` function brings the environment back to its starting point. Resets the cash balance to its initial value (set to \$10,000 in this work), clears all stock holdings, and prepares the initial state by providing the agent with the first window of market data. This ensures that every new training run always begins from a consistent setup.

The `step()` function is where the agent's decisions are applied. The agent sends an array of actions for each stock to the environment. Stocks with negative actions are liquidated by selling any held shares, and the proceeds are stored as cash. The available cash is redistributed across the stocks based on positive allocation signals, proportional to the actions taken.

Table 4.11 demonstrates how the environment executes continuous actions on two consecutive trading days.

TABLE 4.11: Example of Action Execution Across Two Trading Days

Day	Stock	Price	Action	Trade	Holdings (shares)
40	AAPL	71.09	0.09	HOLD	75.87
40	MSFT	159.15	0.79	BUY	32.07
40	GOOGL	65.43	-1.58	SELL	0.00
40	META	184.30	-0.88	SELL	0.00
40	AMZN	96.20	-0.83	HOLD	0.00
Cash = \$0.00, Portfolio Value = \$10,497.38					
41	AAPL	70.14	0.49	HOLD	87.08
41	MSFT	154.65	-0.84	SELL	0.00
41	GOOGL	64.48	0.90	BUY	22.40
41	META	180.24	1.69	BUY	15.14
41	AMZN	95.05	-1.78	HOLD	0.00
Cash = \$0.00, Portfolio Value = \$10,281.48					

On day 40, the agent holds its position in AAPL (75.87 shares), sells its positions in both GOOGL and META, reducing holdings to 0 shares. With the cash available from these sales, the agent allocates the capital based on the action ratio, purchasing 32.07 shares of MSFT at \$159.15, and since no action was taken on AMZN, the shares remain at 0. The portfolio value at the end of day 40 is \$10,497.38, and on day 41, the portfolio value decreases, indicating that the actions chosen did not generate profits. The price of AAPL increased, suggesting that the decision to hold it was beneficial, but the price of MSFT dropped, showing that the action taken on MSFT was not favourable. On day 41, the agent holds AAPL, increasing its holdings to 87.08 shares due to a price drop to \$70.14, sells all its MSFT positions, buys 22.4 shares of GOOGL at \$64.48, buys 15.14 shares of META at \$180.24, and continues to hold its position in AMZN with no shares.

The environment updates the portfolio value based on the latest asset prices after reallocating the capital according to the agent's actions. If additional data is available, the environment proceeds to the next time step and provides the updated state. The environment process ends once all data has been processed. The interaction between the environment and the trading agent is outlined in Algorithm 2, which details how actions are executed, portfolio values are updated, and transitions occur.

Algorithm 2 Stock Market Environment Simulation

```

1: Initialisation: Load historical price data and set initial cash balance  $C_0 = \$ 10,000$ 
2: Set current time step  $t \leftarrow 0$ 
3: Set cash  $C \leftarrow C_0$  and holdings  $h_i \leftarrow 0$  for all assets
4: Set episode status to not done
5: while episode is not done do
6:   for all assets  $i$  do
7:     if action  $a_i < 0$  and holdings  $h_i > 0$  then
8:       Sell all holdings of asset  $i$  at current price  $p_i$ 
9:       Add proceeds to cash:  $C \leftarrow C + h_i \cdot p_i$ 
10:      Set  $h_i \leftarrow 0$ 
11:    end if
12:  end for
13:  Identify assets with positive actions
14:  Compute total sum of positive actions
15:  if positive actions exist and  $C > 0$  then
16:    for all assets  $j$  with  $a_j > 0$  do
17:      Compute allocation ratio:  $w_j = a_j / \sum a_j$ 
18:      Invest: buy shares worth  $w_j \cdot C$  at price  $p_j$ 
19:      Update holdings  $h_j$ 
20:    end for
21:    Set cash  $C \leftarrow 0$ 
22:  end if
23:  Compute portfolio value:  $V_t = C + \sum_i h_i \cdot p_i$ 
24:  Append  $V_t$  to history
25:  Increment timestep:  $t \leftarrow t + 1$ 
26:  if  $t$  reaches end of data then
27:    Set episode status to done
28:  end if
29: end while

```

4.4.6 Full System Integration

Once the PPO agent, neural networks, and custom market environment were fully implemented, the final end-to-end trading system was assembled. This simulation framework operates by looping through the historical dataset one day at a time, executing trades and updating portfolio allocations based on agent decisions.

Running this simulation once produces a complete trajectory of the agent's portfolio value over time. Figures 4.18 and 4.17 show two such portfolio performance examples using the same trained model in two separate evaluation runs. Despite the fact that the model weights are identical, the resulting portfolio trajectories diverge significantly. In the top plot, the agent underperforms the buy-and-hold benchmark, whereas in the bottom plot, it achieves substantial outperformance.

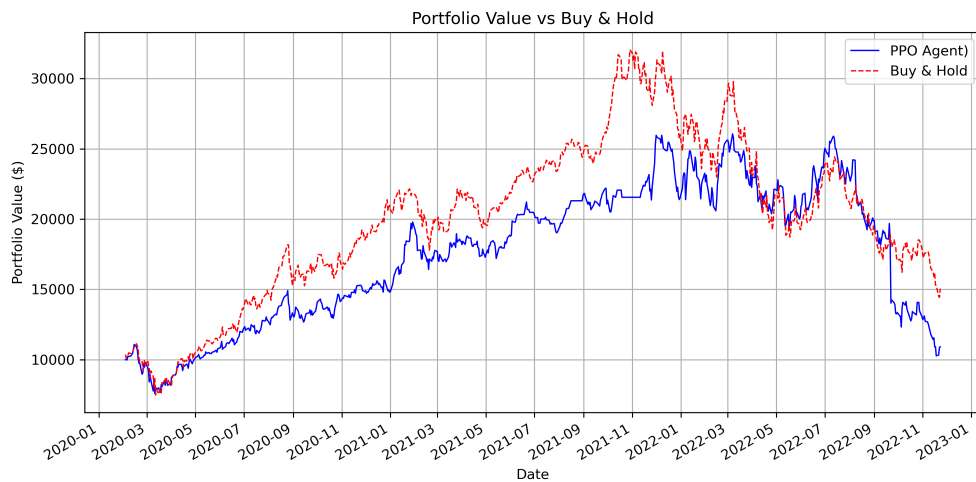


FIGURE 4.17: Under performing portfolio value vs buy-and-hold.

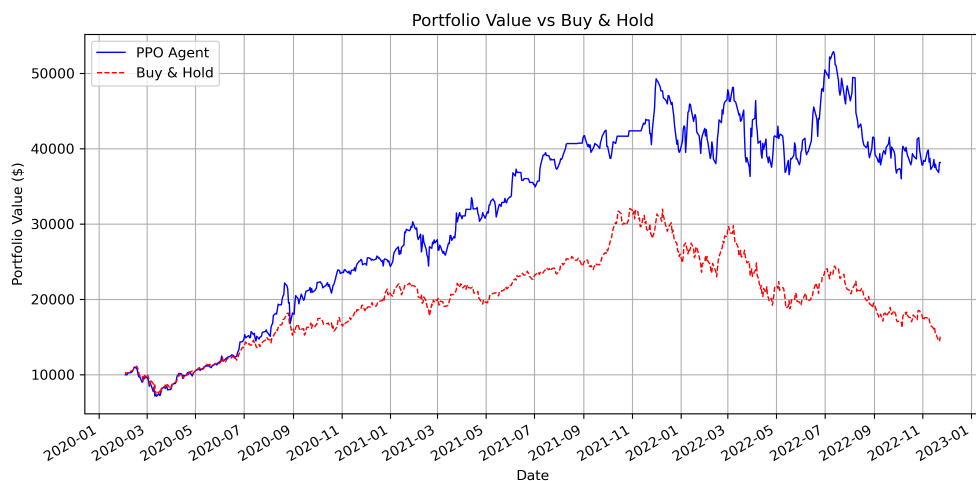


FIGURE 4.18: Over performing portfolio value vs buy-and-hold.

This variation arises because the PPO policy is stochastic as mentioned in Section 4.3.2.2. Even after training, the agent samples actions from the Gaussian probability distribution rather than following a deterministic rule. Hence, each time the environment is run, the agent makes slightly different allocation decisions, even when faced with the same state input and network weights.

These small differences compound over time. Since financial markets are non-linear and path-dependent, even minor divergences in early allocations can lead to very different portfolio compositions later on. For example, a small early trade in TSLA that performs well could enable larger future gains, whereas missing it could lock the agent into underperforming positions. The logic of the environment itself is fully deterministic, but action sampling introduces randomness.

4.5 Conclusion

This behaviour is expected in policy-gradient methods and underscores the importance of evaluating trained agents in multiple runs. To address this, the evaluation strategy in this project incorporates Monte Carlo rollouts, in which the same trained agent is executed repeatedly under different random initialisations. These repeated simulations help capture the range of outcomes that can emerge due to the stochastic behaviour of the agent. Chapter 5 presents the results of these Monte Carlo evaluations, highlighting how different types of neural network architectures and input affect the agent's overall performance and consistency in multiple trading simulations.

Chapter 5

Results and Discussion

This chapter presents and interprets the performance results of the [PPO](#) trading agents trained under varying architectural configurations and input types. Building on the complete system integration described in [Chapter 4](#), the results here are drawn from Monte Carlo evaluations designed to account for the stochastic nature of policy-gradient methods.

5.1 Overview of Evaluation Approach

Each trained agent was evaluated in multiple independent rollouts, using the trained model with the hyperparameters discussed in [Section 4.4](#) to assess not only average performance, but also consistency and stability to random initialisation. In [RL](#), particularly with stochastic policies, small variations in early decision-making can compound over time and lead to significantly different long-term portfolio outcomes.

To account for this, Monte Carlo evaluation was used by running each model 200 times on the training data and 200 times on unseen test data. This allowed for a comprehensive assessment of both in-sample learning and performance under new market conditions. The final portfolio values of all the rollouts were recorded and saved in CSV files, which were used to generate the violin plots presented in [Sections 5.2 and 5.3](#).

5.2 Model Performance on Training Data

[Figure 5.1](#) displays the distribution of the final portfolio values in 200 independent training rollouts for each model and their respective input configuration. The shape of the violin plot reflects both the frequency and concentration of the final results, providing a visual representation of the central tendency and variance in performance. This evaluation

provides insight into how well each architecture was able to learn from the training data and whether it developed a profitable policy. The buy-and-hold strategy produced a final portfolio value of \$17,379 during the same period, representing a **73.79%** return relative to the initial \$10,000 investment.

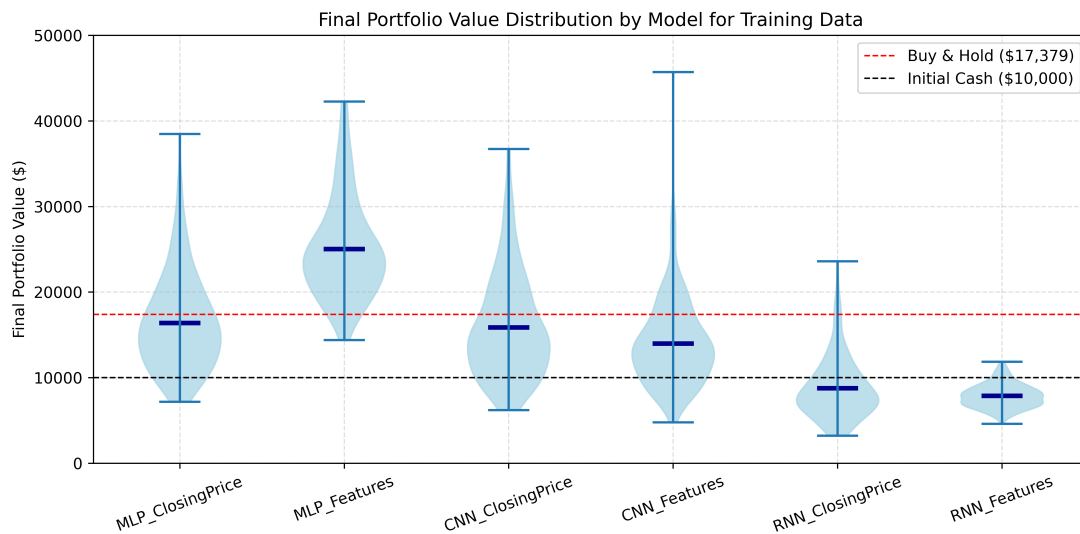


FIGURE 5.1: Violin Plot of the final portfolio values for 200 Monte Carlo rollouts on Training data for each model

To complement the violin plots, Table 5.1 provides a numerical summary of each model's performance, including the average final portfolio value, standard deviation, minimum and maximum outcomes, and the percentage of runs that outperformed the buy-and-hold benchmark. The table also reports the average return relative to the initial investment of \$10,000, offering a quantitative benchmark alongside the visual analysis.

TABLE 5.1: Training-Time Performance Summary Across 200 Monte Carlo Rollouts

Model	Mean (\$)	Std (\$)	Min (\$)	Max (\$)	Beats Buy & Hold
MLP Closing	16,383.73	5,569.86	7,201.31	38,477.27	36%
MLP Features	25,450.46	6,059.43	13,913.23	43,413.63	74%
CNN Closing	15,877.71	6,049.15	6,212.03	36,734.68	32%
CNN Features	13,991.59	5,077.66	4,780.84	45,713.28	21%
RNN Closing	8,781.60	3,563.71	3,238.53	23,595.26	4%
RNN Features	7,883.33	1,367.02	4,611.32	11,860.20	0%

The *MLP Closing Price* model exhibits a moderately compact symmetrical violin shape centred just below the buy-and-hold benchmark. The distribution is narrow at the base and tapers at the top, indicating stable but modest returns across runs. Although not

the best performer, the model demonstrates reasonable consistency despite its simple architecture and limited input, as it still beats the buy-and-hold baseline 36% of the time. This behaviour is probably the result of the early stabilisation of the average log-probability during training, suggesting conservative learning.

The *MLP Features* model achieves the highest median final portfolio value in all configurations. The violin plot shows a broad central mass with short tails, indicating that most runs clustered around high-return values. In particular, the minimum final portfolio value was \$13,913.23, approximately \$5,000 higher than the lowest result of any other model, indicating a significantly reduced risk of capital loss. It is also the only configuration where all rollouts remained above the initial capital and outperformed the buy-and-hold benchmark in 74% of the runs, while all other models fell below 36%. These results underscore the model's ability to extract meaningful patterns from the richer feature set and maintain stable, high-quality policy performance with a relatively simple architecture.

The violin plot for the *CNN Closing Price* model is similar in shape to that of *MLP Closing*, with a slightly narrower middle and comparable tail lengths. Both models show a moderate concentration of results around the median, with the bulk of the outcomes centring just below the buy-and-hold baseline. However, despite the deeper and more expressive structure of CNN, the lack of consistent improved performance relative to the *MLP Closing* baseline suggests an under-utilisation of its capacity. As discussed in Section 4.12, while the actor loss shows early convergence with bounded fluctuations and the log-probability stabilises smoothly, the policy ultimately exhibits only modest learning gains. This points to a model that is stable but potentially limited in extracting meaningful temporal patterns from the closing price input alone.

The violin plot for the *CNN Features* model is wide and irregular in shape, with a noticeably large bulge centred closer to the initial capital baseline and an extended upper tail. This indicates a high degree of variability in final portfolio values, with many runs clustered around lower returns and a few reaching substantially higher outcomes. The minimum final portfolio value was \$4,780.84, while the maximum reached \$45,713.28, reflecting an extremely wide range that indicates unstable and inconsistent policy behaviour. Compared to *CNN Closing Price*, the distribution suggests not only a poorer median performance, but also a greater inconsistency in learning outcomes. The shape reflects the unstable learning dynamics observed during training, where actor loss remained noisy and average log-probability declined steadily throughout, as seen in Section 4.4.2.2, pointing to poor policy calibration and overconfidence in suboptimal actions. Despite having access to a richer feature set, the model fails to

leverage it successfully, probably due to over-fitting and an inability to extract coherent representations from the expanded input space.

The violin plot for the *RNN Closing Price* model is short and compact, with most of its mass concentrated well below the initial capital line, indicating that the majority of rollouts resulted in an average capital loss of -12%. Only 4% of the runs exceeded the buy-and-hold benchmark. This distribution highlights severe underperformance and trading performance. The violin plot confirms the training behaviour observed in Figure 4.15, where early stagnation in actor loss and a reversal in log-probability indicated limited policy improvement and decreasing certainty. The model captures some short-term patterns but lacks the capacity to handle longer temporal dependencies, reducing its suitability for PPO.

Finally, the *RNN Features* model shows the weakest performance overall. The violin plot is short and extremely compact, with all portfolio values clustered below the initial capital baseline. The shape indicates almost no variation across runs and consistent capital loss, with no outliers suggesting a meaningful upside. This outcome is consistent with the training behaviour shown in Figure 4.16, further confirming that the vanilla RNN lacks the capacity to process high-dimensional input effectively under PPO.

5.3 Model Performance on Testing Data

Figure 5.2 presents the distribution of the final portfolio values in 200 test time rollouts for each model. Compared to training distributions, test results show notable changes in both performance consistency and overall return levels. For reference, the buy-and-hold strategy yielded a final portfolio value of \$20,393 during the same period, corresponding to a **103.93%** return relative to the initial investment of \$10,000.

Table 5.2 presents the detailed breakdown of the performance of each model during training.

TABLE 5.2: Testing-Time Performance Summary Across 200 Monte Carlo Rollouts

Model	Mean (\$)	Std (\$)	Min (\$)	Max (\$)	Beats Buy & Hold
MLP Closing	20,054.37	3,360.91	10,578.92	28,597.01	43%
MLP Features	25,272.31	1,949.79	20,653.74	31,169.17	100%
CNN Closing	20,470.83	3,664.16	13,822.76	36,556.45	46%
CNN Features	15,559.80	2,485.96	9,118.41	25,221.02	3.5%
RNN Closing	20,355.62	2,929.30	14,554.14	31,436.22	46%
RNN Features	15,618.16	342.63	14,499.96	16,808.67	0%

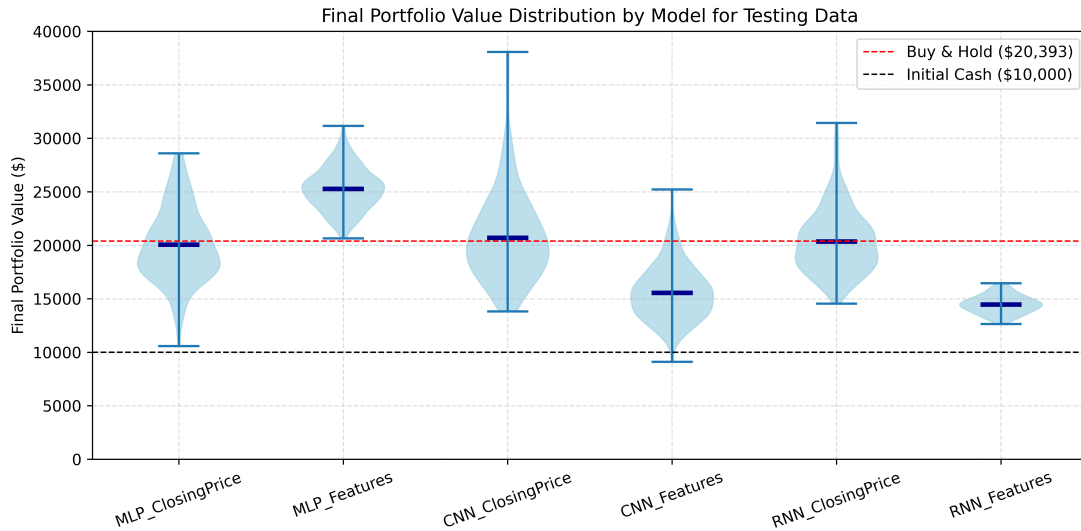


FIGURE 5.2: Violin Plot of the final portfolio values for 200 Monte Carlo rollouts on Testing data for each model

The *MLP Closing* model performs moderately well on the test set, showing an improvement over its training performance. The violin plot is tall and relatively compact, with most of the distribution concentrated between \$18,000 and \$22,000. The median is just below the buy-and-hold benchmark, with 43% of the runs outperforming the buy-and-hold benchmark. The model never produced a final portfolio value below the initial capital, suggesting consistent capital preservation. Although the limited input space may have constrained its ability to fully exploit the 2023 uptrend, the results demonstrate stable performance from a simple architecture.

The violin plot for the *MLP Features* model in the testing data is compact and symmetric, with all values placed above the buy-and-hold benchmark. The mean final portfolio value was \$25,272.31, representing an average excess return of 52.72% over the benchmark. The minimum value, \$20,653.74, further indicates that each run not only preserved capital but also outperformed the buy-and-hold strategy, making this the only model to achieve such consistent returns. The narrow spread and lack of outliers suggest a stable policy that reliably executes across different rollouts, providing clear evidence that the agent successfully learnt a profitable trading strategy.

The violin plot for the *CNN Closing* model is fairly wide, with most of the density centred around the buy-and-hold benchmark, showing a fairly even spread of results. The mean final portfolio value was \$20,470.83, which is only slightly above the benchmark, giving it less than a 1% advantage. However, the upper tail is quite long, with the highest performing run reaching \$36,556.45 which is more than \$5,000 more than the maximum value seen in the *MLP Features* model. This suggests that, under favourable conditions,

the model is capable of generating substantial gains. However, the broader spread and variation across runs point to less consistent learning, indicating that while the CNN occasionally captures profitable signals, it lacks the reliability observed in more stable architectures.

The violin plot for the *CNN Features* model is fairly wide, with a noticeable bulge just above the initial capital line, but still well below the buy-and-hold benchmark. The model achieved an average return of 55.60% and beat the benchmark in only 3.5% of the runs. It is also the only model where some rollouts ended below the initial investment, with a minimum portfolio value of \$9,118.41, indicating that in some cases the agent actually lost money. The distribution suggests that, while the model occasionally identifies useful patterns, it is often misled by the additional features. This outcome reinforces earlier concerns that the CNN architecture struggled to extract reliable representations from the high-dimensional input, likely due to over-fitting or poor policy calibration.

The violin plot for the *RNN Closing* model is relatively compact, with most values clustered around the buy-and-hold benchmark. The mean final portfolio value was \$20,355.62, which is almost identical to the results from the *MLP Closing* and *CNN Closing* models. However, the RNN's distribution shows less variability, with a tighter spread apart from a single high-performing outlier reaching \$31,436.22. This suggests that while the model does not produce extreme gains as often, it delivers more consistent outcomes in comparison to the other two. Despite using a basic recurrent structure, the model appears to have captured some useful short-term patterns in the price data, contributing to relatively steady test-time performance. However, this may be partly attributed to the fact that 2023 was largely an upward trending market, which could have favoured simpler trend-following behaviours.

The violin plot for the *RNN Features* model is extremely narrow and tightly clustered, showing very little variation between runs. The mean final portfolio value was \$15,618.16 with a standard deviation of just \$342.63, and no single rollout outperformed the buy-and-hold benchmark. Despite achieving an average return of 56.18%, all runs remained below the benchmark but still managed to not lose any capital. The lack of spread in the distribution suggests that the model failed to adapt meaningfully to different market conditions. This reinforces the conclusion that vanilla RNNs struggle with a high-dimensional input and are ill-suited for [PPO](#) in financial tasks that require richer temporal and feature-aware representations.

5.3.1 Risk–Reward Classification

Based on the test-time distribution of the final portfolio values in Figure 5.2, each model can be categorised by its relative risk and reward profile. These classifications are summarised in Table 5.3, which reflects both the volatility of the returns and the median performance of each configuration. In practical terms, such trade-offs resemble client-facing investment choices since some stakeholders may prefer stable, lower-risk strategies with moderate gains, while others may tolerate higher volatility for the chance of exceptional returns.

Two standout models emerge depending on the investor’s risk tolerance. The *MLP Features* model demonstrates the most consistent performance, with all rollouts outperforming the buy-and-hold baseline, making it the preferred choice for risk-averse scenarios. In contrast, the *CNN Closing Price* model exhibits high variability but also achieved some of the highest individual returns among all configurations. Given that its worst-case performance is comparable to the *RNN Closing Price* model, but with substantially higher upside potential, it represents a compelling option for higher-risk strategies seeking greater rewards.

TABLE 5.3: Risk–Reward Classification of PPO Models on Testing Data

Model	Risk Level	Reward Level
MLP Closing	Moderate	Moderate
MLP Features	Low	Moderate
CNN Closing	High	High
CNN Features	Moderate	Low
RNN Closing	Moderate	Moderate
RNN Features	Low	Low

5.3.2 Computational Considerations

In addition to performance metrics, the duration of training is a critical consideration when selecting models for real-world deployment. Table 5.4 summarises the total training time and the average time per epoch for each configuration obtained on a MacBook Air with a 1.1 GHz quad-core Intel Core i5 processor. The results highlight the significant variation in computational cost between architectures. Simpler models, such as the *MLP* variants, train much faster than the *CNN* and *RNN* models which take significantly more time. It is also worth noting that the *CNN* and *RNN* models were trained using larger batch sizes than the *MLP* model. If trained with the same batch size of 32 used in the

MLP configurations, their training times would have increased significantly beyond the current measurements. In addition, feature-based models require additional computation for preprocessing and exhibit slower convergence due to their larger input dimensionality.

TABLE 5.4: Training Duration by Model

Model	Epochs	Total Time	Time/Epoch (min)
MLP Closing	200	03:55	1.18
MLP Features	500	05:40	0.68
CNN Closing	700	33:49:00	2.90
CNN Features	500	44:09:00	5.29
RNN Closing	500	38:42:00	4.64
RNN Features	500	51:21:49	6.16

Taking into account both performance and computational efficiency, the *MLP Features* model stands out as the best overall. It consistently delivers high returns with minimal risk, performs well across all test rollouts, and maintains low training time and resource demands. While the *CNN Closing* model offers competitive upside potential for risk-tolerant scenarios, its significantly higher training cost and variability in outcomes make it less suitable for practical deployment where stability and processing speed are prioritised. Therefore, the *MLP Features* configuration represents the best trade-off between performance, consistency, and computational cost.

5.4 Comparison with Supervised Learning Implementation

To conclude the comparative analysis, the median portfolio trajectories of the feature-based PPO models of Figure 5.2 were plotted along with the supervised MLP baseline from Section 4.2 and the buy-and-hold strategy, as seen in Figure 5.3. The *MLP Features* model clearly outperforms all other approaches, steadily increasing in value and significantly outperforming both the buy-and-hold benchmark and the supervised model by the end of the year. The *CNN Features* model also remains competitive, closely tracking the supervised baseline for most of the period, although with more fluctuations, but finishes slightly higher than the supervised MLP.

In contrast, the *RNN Features* model lags behind throughout the period. Its trajectory rises slowly and plateaus early, reinforcing the earlier conclusion that vanilla RNNs struggle to learn in this financial context. Although the supervised model performs well

during strong market trends, achieving a return of approximately 70%, it shows reduced adaptability during volatile periods. In comparison, *MLP Features* exhibits more stable and consistent performance, delivering a return of approximately 155%, which is more than double the return of the supervised MLP. This suggests that RL, when coupled with appropriately chosen architectures, can better exploit sequential feedback in volatile markets.

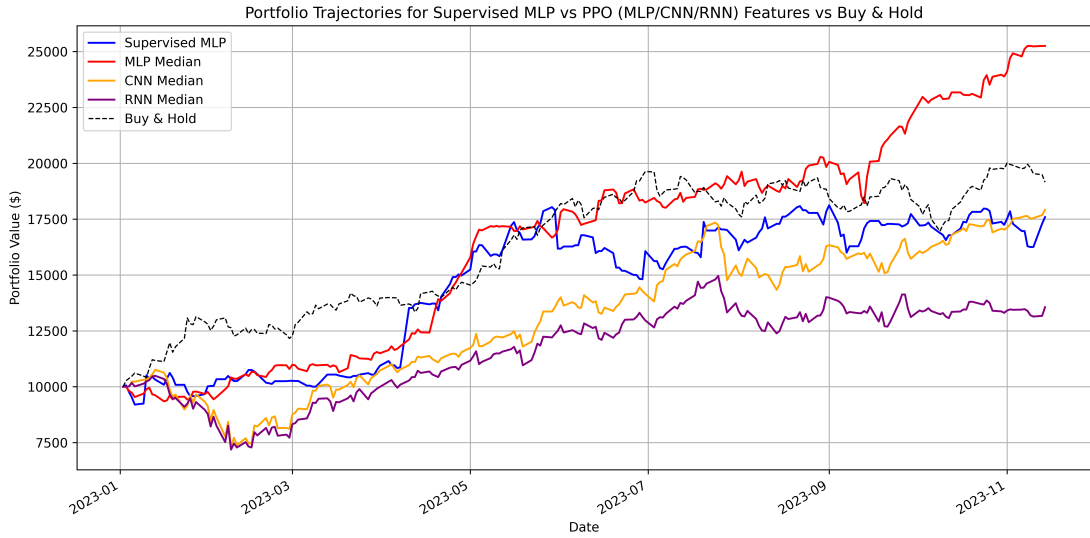


FIGURE 5.3: Median portfolio value trajectories from 200 Monte Carlo runs for PPO agents using MLP, CNN, and RNN architectures with engineered features as input, compared against a supervised MLP baseline and the buy-and-hold strategy during the 2023 test period.

5.5 Conclusion

Across Monte Carlo rollouts in both training and testing, the *PPO* agents revealed clear distinctions in model performance and stability. The *MLP Features* model outperformed all other architectures, achieving strong returns with minimal risk and low computational costs. Although deeper models occasionally produced higher returns, they lacked consistency. Compared to the supervised baseline, well-tuned RL models not only matched expectations but demonstrated greater adaptability in dynamic market conditions.

Chapter 6

Conclusion

This dissertation aimed to explore whether [DRL](#), and more specifically the [PPO](#) algorithm, could be used to identify optimal investment strategies in stock trading. Motivated by the limitations of rule-based systems and traditional supervised learning in financial markets, the project aimed to design an intelligent agent capable of making sequential trading decisions by learning from direct interaction with a simulated market environment.

6.1 Project Overview

A detailed theoretical foundation was established to understand the core principles of RL, beginning with MDP and moving to DRL through the neural function approximation. This guided the selection of PPO over alternative RL algorithms. To better understand the data requirements, a baseline supervised MLP model was first implemented, which also provided a reference point for comparison with RL-based methods.

The dataset was cleaned, normalized, and formatted for time-series modelling, with additional features engineered to reflect meaningful market behaviour. A custom PPO agent was developed, followed by the construction of a modular RL environment capable of supporting multiple stock trading. Three neural network architectures, MLP, CNN and RNN, were used and tuned across two state representations: normalised closing prices and normalised market features. These models were evaluated in 200 Monte Carlo rollouts, with particular attention to their risk-reward characteristics and consistency of performance.

The best performing model was the MLP trained on market features, achieving a mean final portfolio value of \$25,272.31 during the test period for 2023, equivalent to a return of 152.72%. This model outperformed the buy-and-hold strategy in every test case, exhibiting an attractive risk profile with low variance. The CNN model, while

demonstrating the potential for extreme returns of more than 250%, lacked consistency as its mean only beat the benchmark by 1%. The RNN models performed the poorest because of their lack of gating mechanisms, making it difficult for them to retain relevant information over longer sequences and learn from the high-dimensional market data.

Compared to the supervised learning baseline, the PPO-based MLP with features model demonstrated superior adaptability to volatile market conditions, providing more stable growth during drawdowns and achieved more than twice the return of the supervised model. Computationally, the PPO MLP model was also the most efficient, requiring less training time and fewer resources than the CNN and RNN models, which had high computational costs and limited performance gains.

6.2 Future Developments

Although the results demonstrate the potential of DRL in portfolio optimisation, several areas remain open for further exploration. One such area is the resolution of market data used during training and evaluation. This work relied on daily price data, which can obscure short-term fluctuations that are relevant for high-frequency decision-making. Using hourly or minutely data could allow the agent to capture more detailed market dynamics, potentially improving responsiveness to intra-day trends and volatility patterns. Performance was also shaped by the characteristics of the stocks chosen. As seen in Figure 3.1, NVDA and TSLA were considerably more volatile than the rest of the stocks in the portfolio. Removing these high-volatility stocks would have led to more stable returns and better model consistency.

The reward structure used in this project was intentionally kept simple, as discussed in Section 4.3.2.1. Although sufficient for basic learning, the literature offers a range of alternative reward signals that can provide more informative feedback from the environment. For example, incorporating penalties for drawdowns or rewarding volatility-adjusted gains can offer clearer insight into how well the actions performed. These rewards can also be tuned based on the user's risk preference, whether aiming for a more aggressive or conservative strategy. Experimenting with such reward designs allows the environment to deliver richer signals and can help guide the agent towards better tuning of the networks and optimal policy value.

All experiments in this project were carried out using seven large-cap technology stocks. However, the same networks and training pipeline can be adapted to other sectors in the Global Industry Classification Standards, as discussed in Table 3.2. Doing so would expose the PPO agent to different price changes, volatility markets, and sector-specific

trends, allowing for a more thorough evaluation of the agent's performance across varying market environments.

Although two different state representations were used in this project, the interaction between specific market indicators with the different model architectures was not explored. A logical next step would be to tailor the input features to better align with the strengths of each model type. For instance, in the case of the CNN, using only the closing price may not have provided enough information, while including all six features might have introduced unnecessary noise. A more balanced approach, such as combining the closing price with one or two carefully chosen indicators, could improve performance. In contrast, the MLPs handled the six-feature input well and showed strong feature extraction capabilities. Given their fully connected architecture, MLPs could benefit from the inclusion of higher-level inputs such as sentiment scores derived from financial news or macroeconomic indicators such as interest rates and inflation.

Although PPO includes an entropy bonus to encourage exploration, this feature was not used in the current implementation as discussed in Section 2.4.1.4. Incorporating and tuning the entropy coefficient could help prevent premature convergence by encouraging the agent to explore a broader range of policies, especially during the early stages of training, making sure an optimal policy is found.

6.3 Future Research

Beyond the current implementation, several research directions can be explored to more thoroughly evaluate the performance of DRL in portfolio optimisation. As discussed in Section 4.4.3, the vanilla RNN architectures used in this project exhibited issues with learning stability and overall performance. A natural next step would be to further tune the PPO algorithm and replace the vanilla RNNs with the [LSTM](#) or [GRU](#) networks for both the actor and the critic.

Another future improvement could be to implement a standardised environment using established libraries such as Gymnasium or FinRL, as mentioned in Section 4.4.5. This would involve replacing the custom setup with one that follows widely adopted [APIs](#), improving compatibility with other RL tools, and easing integration with more agent workflows. Switching to these frameworks would require re-tuning the PPO algorithm and adapting the training pipeline to match their environment structure and data handling.

In this study, PPO was the main algorithm used. However, other actor-critic methods, both off-policy and on-policy, which were discussed in Section 2.5.5, can be tested and

compared. Running these algorithms on the same model architectures would show how each one performs under the same conditions and which gives the best results.

A long-term extension of this work could involve connecting the agent to real-time market data and executing simulated trades through a broker [API](#) such as Alpaca or Interactive Brokers. This setup would introduce real-world factors such as latency, slippage, and liquidity constraints, allowing for a more realistic assessment of how the model performs under actual market conditions. Together, these directions offer a clear path towards better understanding the practical impact of DRL in financial markets.

6.4 Final Remarks

RL has shown strong capabilities in strategic decision-making tasks, notably outperforming human experts in games such as chess. Although such environments are rule-bound and fully observable, financial markets present a much greater challenge due to their inherent volatility, partial observability, and sensitivity to external factors. Despite these differences, both domains share the core problem of making informed decisions under uncertainty. This project has shown that DRL is capable of learning from historical market data, adapting to dynamic conditions, and generating trading strategies that are not only viable but competitive with conventional methods.

Although the stock market remains inherently difficult to predict, the results presented in this study suggest that DRL provides a structured and adaptive approach to navigate this uncertainty. The strong performance of the PPO-based MLP model when trained on engineered market features demonstrates the potential of RL to serve as a practical tool in algorithmic trading. Although there is no direct benchmark against professional human trader decisions, the models developed in this project are capable of generating consistent returns with minimal human input. Overall, this project demonstrates that DRL, when combined with the appropriate model architecture and well-designed input features, can outperform both passive strategies and traditional supervised learning approaches in simulated trading environments. With further research, DRL has the potential to consistently outperform the best traders in the industry.

References

- [1] Phung, and Rhee,. A high-accuracy model average ensemble of convolutional neural networks for classification of cloud image patches on small datasets. *Applied Sciences*, 9:4500, 10 2019. doi: 10.3390/app9214500.
- [2] GeeksforGeeks,. Introduction to recurrent neural network, 2023. URL <https://www.geeksforgeeks.org/introduction-to-recurrent-neural-network/>. Accessed: March 4, 2025.
- [3] Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. Number 9. Trends in Cognitive Sciences, 1999. doi: 10.1016/s1364-6613(99)01331-5.
- [4] Quantified Strategies,. What percentage of trading is algorithmic?, 2024. URL <https://www.quantifiedstrategies.com/what-percentage-of-trading-is-algorithmic/>. Accessed: 2025-05-10.
- [5] Financial Times,. Algorithms prop up the market as fretful humans sit out the uncertainty. *Financial Times*, 2023. URL <https://www.ft.com/content/1c359259-5c29-4be6-9cb3-776dbacd0f70>. Accessed: 2025-05-10.
- [6] Qiang, W. and Zhongli, Z. Reinforcement learning model, algorithms and its application. In *2011 International Conference on Mechatronic Science, Electric Engineering and Computer (MEC)*, pages 1143–1146, 2011. doi: 10.1109/MEC.2011.6025669.
- [7] Naeem, M., Rizvi, S. T. H., and Coronato, A. A gentle introduction to reinforcement learning and its application in different fields. *IEEE Access*, 8:209320–209344, 2020. doi: 10.1109/ACCESS.2020.3038605.
- [8] Pecioski, D., Gavriloski, V., Domazetovska, S., and Ignjatovska, A. An overview of reinforcement learning techniques. In *2023 12th Mediterranean Conference on Embedded Computing (MECO)*, pages 1–4, 2023. doi: 10.1109/MECO58584.2023.10155066.
- [9] McGowan, M. J. The rise of computerized high frequency trading: Use and controversy. *Duke Law & Technology Review*, 9:1–25, 2010. URL <https://www.dukelawreview.org/volume-9/issue-1/article-1>.

- [//scholarship.law.duke.edu/dltr/vol9/iss1/15/](https://scholarship.law.duke.edu/dltr/vol9/iss1/15/). Accessed: 2025-05-10.
- [10] TrendSpider, D. e. shaw & company, 2024. URL <https://trendspider.com/learning-center/d-e-shaw-company/>.
- [11] Ibrahim, R. and Al-Haija, Q. Fundamentals of machine learning models. *ResearchGate*, 2025. doi: 10.1201/9781003520498-1.
- [12] Li, H., Lin, L., and Zeng, H. *Machine Learning Methods*. Springer Nature Singapore, Singapore, 1st edition, 2024.
- [13] S, K., S, V., and R, R. A comparative analysis on linear regression and support vector regression. In *2016 Online International Conference on Green Engineering and Technologies (IC-GET)*, pages 1–5, 2016. doi: 10.1109/GET.2016.7916627.
- [14] Sushanth, T., Siddarda, T. S., Sathvika, A., Sayee Shruthi, A. S., Lekha, A., and Kumar, T. S. Time series forecasting using rnn. *Indian Scientific Journal Of Research In Engineering And Management*, 08 (11):1–8, November 2024. doi: 10.55041/ijsrem39164. URL <https://ijsrem.com/download/time-series-forecasting-using-rnn/?wpdmdl=40557&refresh=6748a77d473c31732814717>.
- [15] Delvadiya, J., Varshney, N., Garde, Y., and Shrivastava, A. Unleashing the power of machine learning: cutting-edge innovations and real-world applications. *ResearchGate*, March 2024. doi: 10.58532/v3bact5p4ch4.
- [16] Anyango, W. O. and Gichaiya, M. W. *Exploring The Impact Of Machine Learning On Financial Markets: Opportunities, Risks, And Regulatory Challenges*, 2024. URL <https://www.iosrjournals.org/iosr-jhss/papers/Vol.29-Issue11/Ser-5/F2911053139.pdf>. Accessed on 12 February 2025.
- [17] Xie, N. and Zhang, W. Empowering big data analytics through machine learning: Applications, challenges, and future directions. *Applied and Computational Engineering*, 93(1):76–82, November 2024. doi: 10.54254/2755-2721/93/20240917. URL <https://www.ewadirect.com/proceedings/ace/article/view/16570/pdf>.
- [18] Dridi, S. Reinforcement learning - a systematic literature review. *OSF Preprints*, December 2021. doi: 10.31219/osf.io/svykj. URL <https://osf.io/svykj/download>.
- [19] Perros, H. G. *Support Vector Machines*, page 279–302. Chapman and Hall/CRC, March 2021. doi: 10.1201/9781003139041-11.
- [20] Lo, S.-C., Lin, C.-C., and Chuang, Y.-C. Using support vector machine and sequential pattern mining to construct financial prediction model. In *2008*

- IEEE Asia-Pacific Services Computing Conference*, pages 993–998, 2008. doi: 10.1109/APSCC.2008.190.
- [21] Sharda, R., Delen, D., and Turban, E. *Business Intelligence, Analytics, and Data Science: A Managerial Perspective*. Pearson, 4th edition, 2020.
- [22] Goodfellow, I., Bengio, Y., and Courville, A. *Deep Learning*. MIT press, 2016.
- [23] Aggarwal, C. C. *Neural Networks and Deep Learning: A Textbook*. Springer, 2018.
- [24] Schmidhuber, J. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015. doi: 10.1016/j.neunet.2014.09.003.
- [25] Gabri  , M., Ganguli, S., Lucibello, C., and Zecchina, R. Neural networks: from the perceptron to deep nets. *arXiv preprint arXiv:2304.06636*, 2023. URL <https://arxiv.org/abs/2304.06636>.
- [26] Nielsen, M. A. Neural networks and deep learning. *Determination press*, 2015.
- [27] LeCun, Y., Bengio, Y., and Hinton, G. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [28] Hinton, G. E. and Salakhutdinov, R. R. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.
- [29] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [30] Ioffe, S. and Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, pages 448–456, 2015.
- [31] Pan, S. J. and Yang, Q. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010.
- [32] Hornik, K., Stinchcombe, M., and White, H. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.
- [33] Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- [34] Glorot, X., Bordes, A., and Bengio, Y. Deep sparse rectifier neural networks. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 15 of *Proceedings of Machine Learning Research*, pages 315–323. JMLR Workshop and Conference Proceedings, 2011.

- [35] Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- [36] Kostrikov, I., Yarats, D., and Fergus, R. What matters in on-policy reinforcement learning? a large-scale empirical study. In *International Conference on Learning Representations (ICLR)*, 2020. URL <https://openreview.net/forum?id=r1etN1rtPB>.
- [37] LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [38] Schulman, J., Wolski, F., Dhariwal, P., and others,. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [39] Lipton, Z. C., Berkowitz, J., and Elkan, C. A critical review of recurrent neural networks for sequence learning. *arXiv preprint arXiv:1506.00019*, 2015. URL <https://arxiv.org/abs/1506.00019>.
- [40] Mar, T. Insight into lstm, 2023. URL https://thorirmar.com/post/insight_into_lstm/. Accessed: March 4, 2025.
- [41] Song, X., Chen, K., Bi, Z., Niu, Q., Liu, J., Peng, B., Zhang, S., Liu, M., Li, M., and Yuan, Z. Mastering reinforcement learning: Foundations, algorithms, and real-world applications, February 2025. URL https://osf.io/preprints/osf/bg79j_v2. Preprint.
- [42] Singh, V., Chen, S.-S., Singhania, M., Nanavati, B., Kard, A., and Gupta, A. *How are reinforcement learning and deep learning algorithms used for big data-based decision making in financial industries - A review and research agenda*, 2023.
- [43] Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Driessche, G. V. D., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., and Hassabis, D. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. doi: 10.1038/nature16961.
- [44] Kaelbling, L. P., Littman, M. L., and Moore, A. W. Reinforcement learning: A survey. *Journal of Artificial Intelligence Research*, 4:237–285, 1996.
- [45] Awad, A. L., Elkaffas, S. M., and Fakhr, M. W. Stock market prediction using deep reinforcement learning. *Applied System Innovation*, 6(106), 2023. doi: 10.3390/asi6060106. URL <https://www.mdpi.com/2571-5577/6/6/106>.
- [46] Puterman, M. L. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 2014.

- [47] Feinberg, E. A. and Schwartz, A. *Handbook of Markov Decision Processes: Methods and Applications*. Springer, Boston, MA, 2012.
- [48] Ross, S. M. *Introduction to Probability Models*. Academic Press, 11th edition, 2014.
- [49] van Otterlo, M. and Wiering, M. Reinforcement learning and markov decision processes. *Reinforcement Learning: State-of-the-Art*, pages 3–42, 2012. doi: 10.1007/978-3-642-27645-3_1.
- [50] Mnih, V., Kavukcuoglu, K., Silver, D., and others,. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [51] Peng, D., Gui, Z. W., and Wu, H. Interpreting the curse of dimensionality from distance concentration and manifold effect. *arXiv.org*, abs/2401.00422, December 2023. doi: 10.48550/arxiv.2401.00422.
- [52] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- [53] Watkins, C. J. C. H. and Dayan, P. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992. doi: 10.1007/BF00992698.
- [54] Grondman, I., Busoniu, L., Lopes, G. A. D., and Babuska, R. A survey of actor-critic reinforcement learning: Standard and natural policy gradients. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 42(6):1291–1307, 2012. doi: 10.1109/TSMCC.2012.2218595. URL <https://ieeexplore.ieee.org/document/6392457/>.
- [55] van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30, 2016. URL <https://www.aaai.org/ocs/index.php/AAAI/AAAI16/paper/view/12389>.
- [56] Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., and de Freitas, N. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48, pages 1995–2003. PMLR, 2016. URL <https://proceedings.mlr.press/v48/wangf16.html>.
- [57] Sutton, R. S., McAllester, D., Singh, S., and Mansour, Y. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, pages 1057–1063, 1999. URL <https://papers.nips.cc/paper/1713-policy-gradient-methods-for-reinforcement-learning-with-function-approximation.pdf>.

- [58] Peters, J. and Bagnell, J. A. Policy gradient methods. *Scholarpedia*, 5(11): 3698, 2010. URL https://is.mpg.de/fileadmin/user_upload/files/publications/Peters_EOMLA_submitted_6074%5B0%5D.pdf.
- [59] Jia, Y. and Zhou, X. Y. Policy gradient and actor-critic learning in continuous time and space: Theory and algorithms. *arXiv preprint arXiv:2111.11232*, 2021. URL <https://arxiv.org/abs/2111.11232>.
- [60] Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., and Riedmiller, M. Deterministic policy gradient algorithms. In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, pages 387–395, 2014.
- [61] Chang, J. *Portfolio Allocation Reinforcement Learning PPO Model (Part II)*, August 2023. URL <https://medium.com/@changjulian17/portfolio-allocation-reinforcement-learning-ppo-model-part-ii-56ecf76a94> Accessed on 12 November 2024.
- [62] Liang, X., Yang, X., Miao, Y., and Zhang, J. Ptr-ppo: Proximal policy optimization with prioritized trajectory replay. *arXiv preprint arXiv:2112.03798*, 2021. URL <https://arxiv.org/abs/2112.03798>.
- [63] Mnih, V., Badia, A. P., Mirza, M., and others,. Asynchronous methods for deep reinforcement learning. *International Conference on Machine Learning (ICML)*, pages 1928–1937, 2016.
- [64] Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *arXiv preprint arXiv:1801.01290*, 2018. URL <https://arxiv.org/abs/1801.01290>.
- [65] Busoniu, L., Babuska, R., Schutter, B. D., and Ernst, D. Reinforcement learning and dynamic programming using function approximators. *CRC Press*, 2010.
- [66] L.P., B. Bloomberg terminal. <https://www.bloomberg.com/professional/solution/bloomberg-terminal/>, 2023. Accessed: 2025-05-10.
- [67] TickData, L. Tickdata historical market data solutions. <https://www.tickdata.com/>, 2023. Accessed: 2025-05-10.
- [68] Refinitiv,. Refinitiv market data solutions. <https://www.refinitiv.com/en>, 2023. Accessed: 2025-05-10.
- [69] Yahoo Finance,. Yahoo finance - stock market live, quotes, business & finance news. <https://finance.yahoo.com/>, 2023. Accessed: 2025-05-10.
- [70] Liu, Y., Zhang, X.-Y., Yang, S., Wang, J., Tan, M., Liang, Y., Ren, Z., Wang, Z., Liu, Z., and others,. Finrl: A deep reinforcement learning library for automated

- stock trading in quantitative finance. *arXiv preprint arXiv:2011.09607*, 2021. URL <https://github.com/AI4Finance-Foundation/FinRL>.
- [71] Miller, A. The pitfalls of using free financial data sources. <https://towardsdatascience.com/the-pitfalls-of-using-free-financial-data-sources-2c3a7cceed86>, 2020. Accessed: 2025-05-10.
- [72] Alpha Vantage Inc.,. Alpha vantage api documentation. <https://www.alphavantage.co/documentation/>, 2023. Accessed: 2025-05-10.
- [73] Zhang, Z., Zohren, S., and Roberts, S. Deep reinforcement learning for trading. *arXiv preprint arXiv:1911.10107*, 2019. URL <https://arxiv.org/abs/1911.10107>.
- [74] Smith, J. and Lee, A. The post-pandemic tech rally: Drivers of growth in the digital sector. *Journal of Financial Markets and Innovation*, 18(2):123–139, 2021. Analysis of technology stock performance from 2020 to 2021.
- [75] Murphy, J. J. *Technical Analysis of the Financial Markets: A Comprehensive Guide to Trading Methods and Applications*. New York Institute of Finance, 1999.
- [76] Wilder, J. W. *New Concepts in Technical Trading Systems*. Trend Research, 1978.
- [77] Appel, G. *Technical Analysis: Power Tools for Active Investors*. FT Press, 2005.
- [78] Bollinger, J. *Bollinger on Bollinger Bands*. McGraw-Hill, 2001.
- [79] Liang, Z., Chen, H., Zhu, J., Jiang, K., and Li, Y. Adversarial deep reinforcement learning in portfolio management. *arXiv preprint arXiv:1808.09940*, 2018. URL <https://arxiv.org/abs/1808.09940>.
- [80] Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., and others.,. Tensorflow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 265–283. USENIX Association, 2016.
- [81] Chollet, F. Keras. <https://github.com/keras-team/keras>, 2015.
- [82] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., and others.,. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- [83] Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., and Willing, C. Jupyter notebooks – a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, 2016. Lozana, Switzerland: IOS Press.

- [84] Liu, D. C. and Nocedal, J. On the limited memory bfgs method for large scale optimization. *Mathematical Programming*, 45(1):503–528, 1989.
- [85] Ng, A. Y. Feature selection, l1 vs. l2 regularization, and rotational invariance. In *Proceedings of the twenty-first international conference on Machine learning*, page 78. ACM, 2004.
- [86] Doe, J. and Smith, A. Strategic discounting of future rewards in financial reinforcement learning. *Journal of Financial AI Research*, 12(3):45–60, 2023. Best practices for setting discount factors in trading agents.
- [87] Ulyanov, D., Vedaldi, A., and Lempitsky, V. Instance normalization: The missing ingredient for fast stylization. In *arXiv preprint arXiv:1607.08022*, 2016.
- [88] Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [89] Ba, J. L., Kiros, J. R., and Hinton, G. E. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [90] Bottou, L. Large-scale machine learning with stochastic gradient descent. pages 177–186, 2010.
- [91] Foundation, T. F. Gymnasium: A standard api for reinforcement learning environments. <https://github.com/Farama-Foundation/Gymnasium>, 2023. Accessed: 2025-05-10.
- [92] V, K., G, D., Sarveshwaran, V., Jayakanth, J., M, K., and Renu, Y. Comparative analysis of diverse classification algorithms of machine learning by using various quality metrics. In *2023 IEEE 5th International Conference on Cybernetics, Cognition and Machine Learning Applications (ICCCMLA)*, 2023. doi: 10.1109/ICCCMLA58983.2023.10346683.
- [93] Ward, R. *Stochastic gradient descent: where optimization meets machine learning*, page 5140–5153. EMS Press, December 2023. doi: 10.4171/icm2022/171.
- [94] Singh, S. K., Sharma, T., Santosh, K., Reddy, K., Swagatha, J., and Saravanakumar, R. Utilizing deep neural networks for portfolio optimization in financial markets. In *2024 International Conference on Intelligent Systems and Advanced Applications (ICISAA)*, pages 1–5, 2024. doi: 10.1109/ICISAA62385.2024.10829330.
- [95] Nwadiugwu, M. C. Neural networks, artificial intelligence and the computational brain. *arXiv preprint arXiv:2101.08635*, 2020. URL <https://arxiv.org/abs/2101.08635>.
- [96] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

- [97] Heaton, J. B., Polson, N. G., and Witte, J. H. Deep learning for finance: Deep portfolios. *Applied Stochastic Models in Business and Industry*, 33(1):3–12, 2017.
- [98] Bertsekas, D. P. Dynamic programming and optimal control. In *Proceedings of the 34th IEEE Conference on Decision and Control*, volume 4, pages 4464–4465. IEEE, 1995.
- [99] He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.
- [100] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734. Association for Computational Linguistics (ACL), 2014. doi: 10.3115/v1/D14-1179.
- [101] Deng, Y., Bao, F., Kong, Y., Ren, Z., and Dai, Q. Deep direct reinforcement learning for financial signal representation and trading. *IEEE Transactions on Neural Networks and Learning Systems*, 28(3):653–664, 2016. doi: 10.1109/TNNLS.2016.2522401.
- [102] Lapan, M. *Deep Reinforcement Learning Hands-On*. Packt Publishing, Birmingham, UK, 1st edition, 2018. ISBN 9781788834247.
- [103] Ye, H., Xu, Y., and Wu, Q. Reinforcement learning for portfolio management: A survey. *arXiv preprint arXiv:2010.01065*, 2020.
- [104] Li, X., Jiang, P., Xu, T., and Jin, X. Deep reinforcement learning in portfolio management: A survey. *arXiv preprint arXiv:2008.09940*, 2020.
- [105] Escudero, G. and Padró, L. Challenges of reinforcement learning in financial markets: Pitfalls and best practices. *arXiv preprint arXiv:2306.09321*, 2023.